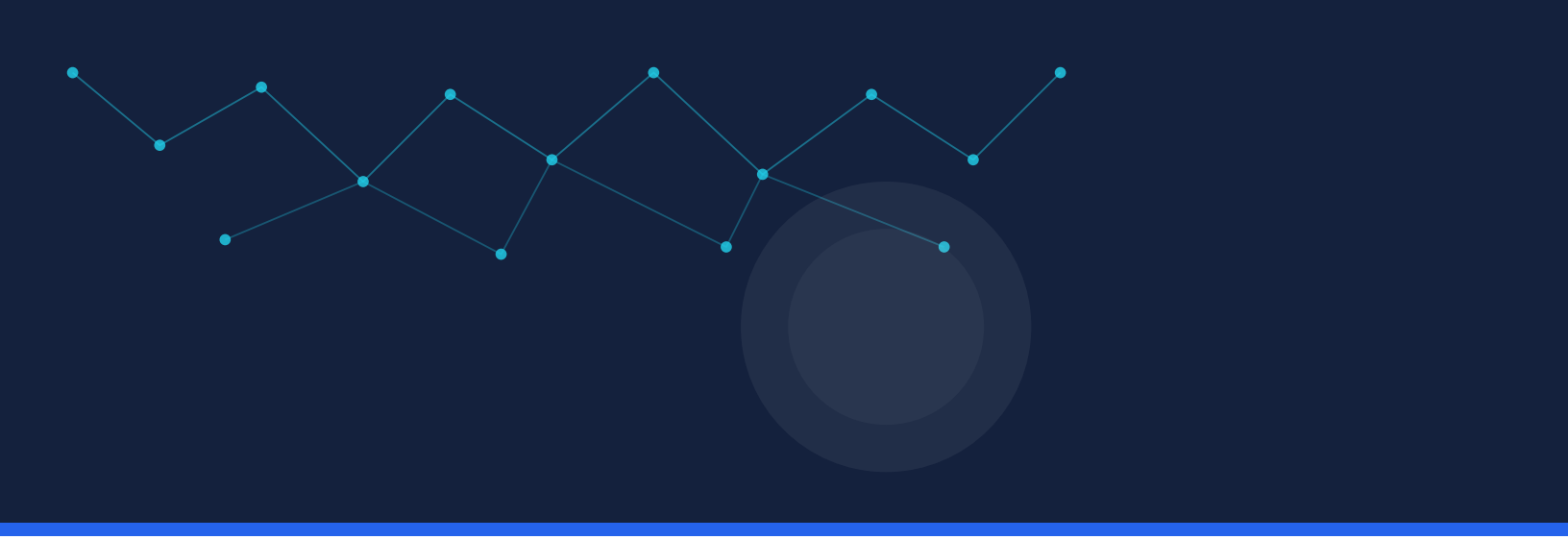


# Clawsha


**Integrantes**

*Laura Andrea Castrillon Fajardo*
*Samuel Martinez Arteaga*
*Jeronimo Campuzano Castaño*

Universidad EAFIT
Medellín, Colombia


Octubre de 2026

# Índice

**1. Objetivo y marco teórico breve** **1**
1.1. Objetivo . . . 1
1.2. Marco teórico . . . 1

**2. Descripción del servicio y del problema abordado** **1**
2.1. Descripción general del servicio . . . 2
2.2. Problema abordado . . . 2
2.3. Requerimientos funcionales y no funcionales . . . 3

**3. Arquitectura del sistema y diagramas** **7**
3.1. Estilo y decisiones arquitectónicas . . . 7
3.2. Componentes del sistema . . . 8
3.3. Diagramas de arquitectura y despliegue . . . 8
3.4. Flujos principales de operación . . . 9

**4. Especificaciones del protocolo y APIs** **12**
4.1. Protocolos de comunicación . . . 13
4.2. Especificación de las APIs . . . 13
4.3. Autenticación, autorización y seguridad . . . 16

**5. Algoritmo de particionamiento y distribución** **17**
5.1. Estrategia de particionamiento . . . 17
5.2. Algoritmo de distribución . . . 19
5.3. Reconstrucción y recuperación de archivos . . . 21
5.4. Replicación, consistencia y tolerancia a fallos . . . 22

**6. Descripción del entorno nativo o Docker** **23**
6.1. Infraestructura y topología de despliegue . . . 23
6.2. Ejecución nativa o contenedorizada . . . 23
6.3. Instalación, configuración y puesta en marcha . . . 24
6.4. Variables de entorno, puertos y dependencias . . . 25
6.5. Puertos . . . 27

**7. Descripción del entorno nativo o Docker** **27**
7.1. Infraestructura y topología de despliegue . . . 27

7.2. Ejecución nativa o contenedorizada . . . . . 28
7.3. Instalación, configuración y puesta en marcha . . . . . 28
7.4. Variables de entorno . . . . . 29
7.5. Puertos . . . . . 29

**8. Pruebas y análisis de resultados** **31**
8.1. Plan y metodología de pruebas . . . . . 31
8.2. Pruebas funcionales . . . . . 32
8.3. Pruebas de rendimiento y escalabilidad . . . . . 33
8.4. Pruebas de disponibilidad, consistencia y seguridad . . . . . 36
8.5. Análisis de resultados y limitaciones . . . . . 38

**9. Conclusiones** **39**

**Referencias** **40**

# 1 Objetivo y marco teórico breve

## 1.1 Objetivo

Diseñar e implementar un sistema de archivos distribuido (Clawsha) que permita almacenar, transferir y recuperar archivos de gran tamaño mediante su distribución entre múltiples nodos. El sistema busca ofrecer una solución que considere la alta disponibilidad, el rendimiento, la escalabilidad, la seguridad y la transparencia en el acceso a los datos. Para ello, se plantea una arquitectura distribuida en la que los componentes colaboran para gestionar los metadatos, localizar los bloques de información y atender las solicitudes de los clientes, procurando mantener la continuidad del servicio ante posibles fallos en la infraestructura.

## 1.2 Marco teórico

Un sistema de archivos distribuido (Distributed File System, DFS) permite almacenar y acceder a archivos cuyos datos se encuentran distribuidos entre diferentes nodos conectados mediante una red. Este tipo de sistemas facilita el acceso concurrente a la información y permite aprovechar los recursos de múltiples máquinas para mejorar la capacidad de almacenamiento, la disponibilidad y el rendimiento. Entre los ejemplos mencionados en el enunciado se encuentran NFS (Network File System), AFS (Andrew File System) y SMB (Server Message Block).

De acuerdo con el mecanismo de almacenamiento y acceso, los sistemas de archivos distribuidos pueden clasificarse, de manera general, en sistemas basados en bloques y sistemas basados en objetos. En los primeros, los archivos se dividen en bloques que pueden almacenarse en diferentes nodos, permitiendo realizar operaciones de lectura y escritura sobre unidades de información más pequeñas. En los sistemas basados en objetos, la unidad de acceso se presenta normalmente como un archivo completo, mientras que la infraestructura interna puede utilizar mecanismos de particionamiento para distribuir sus datos.

El diseño de un DFS debe considerar aspectos como la escalabilidad, la disponibilidad, la consistencia de los datos, la concurrencia y la seguridad. La distribución de los archivos entre varios nodos permite aprovechar la infraestructura disponible, mientras que la replicación puede contribuir a reducir el riesgo de pérdida de información y a mantener el servicio ante fallos. Sin embargo, estas estrategias requieren mecanismos que permitan gestionar los metadatos, localizar los datos y mantener la consistencia de la información.

Finalmente, la arquitectura de un sistema distribuido puede seguir un modelo cliente-servidor, en el que los clientes realizan solicitudes a un servicio compuesto por diferentes nodos, o un modelo entre pares (P2P), en el que distintos participantes colaboran en la prestación del servicio. La comunicación entre los componentes puede implementarse mediante protocolos como HTTP REST, gRPC o sockets TCP, según las necesidades del sistema. Estos conceptos constituyen la base para el diseño de Clawsha y para comprender las decisiones arquitectónicas adoptadas en su implementación.

# 2 Descripción del servicio y del problema abordado

## 2.1 Descripción general del servicio

Clawha es un sistema de archivos distribuido por bloques, inspirado en la arquitectura de HDFS, que ofrece a sus usuarios un espacio personal de almacenamiento accesible desde un navegador web. Cada usuario se registra, inicia sesión y gestiona su propio árbol de directorios: puede crear carpetas, subir, descargar, previsualizar, mover, renombrar y eliminar archivos. Los usuarios están aislados entre sí: la ruta de otro usuario simplemente no existe para el que consulta.

Internamente, cada archivo se divide en bloques de 16 MiB que se almacenan con factor de replicación 2, con cada réplica en una zona de disponibilidad (AZ) distinta. El sistema se compone de cuatro tipos de componentes:

- **Master** (equivalente al *NameNode* de HDFS): plano de control. Mantiene el espacio de nombres (directorios, archivos y sus versiones) en Amazon DynamoDB, decide en qué *workers* se ubica cada bloque y publica las subidas de forma atómica. Nunca manipula el contenido de los archivos. Se despliegan dos instancias en modo activo–pasivo: solo la que posee el *lease* de líder atiende peticiones.
- **Worker** (equivalente al *DataNode*): almacena los bloques en disco, verifica su integridad con SHA-256, informa periódicamente al líder qué bloques tiene (*heartbeats* y *block reports*) y ejecuta las órdenes de replicación.
- **Gateway**: API REST sin estado que autentica a los usuarios con JWT, divide los archivos en bloques, los cifra y los transfiere directamente a los workers mediante gRPC, sin pasar los datos por el master.
- **Frontend**: aplicación web en React con la que interactúa el usuario final.

El backend está escrito en Rust (`tokio`, `tonic`, `axum`) y la comunicación entre componentes internos usa gRPC con un contrato único definido en Protocol Buffers. La infraestructura se despliega en AWS sobre dos zonas de disponibilidad mediante Terraform.

## 2.2 Problema abordado

Un servidor de archivos centralizado presenta tres limitaciones: su capacidad está acotada por el disco de una sola máquina, su caída deja el servicio completamente indisponible y una falla del disco implica la pérdida definitiva de los datos. Clawsha aborda el problema de **almacenar archivos de forma confiable sobre infraestructura que falla**, es decir, ofrecer un servicio de archivos que siga disponible y no pierda datos cuando fallan nodos individuales, discos o incluso una zona de disponibilidad completa.

Concretamente, el sistema debe resolver:

- **Durabilidad**: ningún archivo confirmado se pierde por la caída de un único worker o de una AZ, lo que exige replicar cada bloque en ubicaciones independientes y restaurar automáticamente las copias que se pierden.
- **Disponibilidad del plano de control**: la caída del master no puede detener el servicio, lo que exige un mecanismo de elección de líder y de *failover* automático.
- **Consistencia**: con dos masters existe el riesgo de *split-brain*, es decir, que un líder antiguo que no sabe que perdió el liderazgo corrompa la metadata. Además, un lector nunca debe

observar un archivo a medio escribir.

- **Integridad**: la corrupción silenciosa de datos en disco debe detectarse antes de entregarse al usuario y antes de que se pierda la última copia sana.
- **Escalabilidad**: la capacidad de almacenamiento debe crecer agregando workers, sin redistribuir manualmente los datos.

El alcance se limita a un único clúster en una región de AWS, con semántica de escritura de archivo completo (*write-once* por versión, sin escrituras parciales ni concurrentes sobre un mismo archivo) y usuarios autenticados con usuario y contraseña.

## 2.3 Requerimientos funcionales y no funcionales

### Requerimientos funcionales

Se agrupan por componente: lo que ve el usuario, lo que hace el plano de control (master) y lo que garantiza cada nodo de almacenamiento (worker).

| ID | Requerimiento |
|---|---|
| **Servicio al usuario** | |
| RF1 | Registro e inicio de sesión de usuarios. Las contraseñas tienen al menos 8 caracteres, se almacenan con *argon2id* y el sistema emite un JWT con vigencia de una hora. No pueden existir dos usuarios con el mismo nombre. |
| RF2 | Operaciones sobre el espacio de nombres de cada usuario: listar un directorio, crear directorios, eliminar archivos y directorios (recursivamente si no están vacíos), mover o renombrar entradas y consultar sus metadatos. |
| RF3 | Bloqueo de escritura por ruta: mientras un cliente escribe un archivo, ningún otro puede escribir en la misma ruta. El bloqueo es un *lease* de 60 s que se renueva mientras dura la subida. |
| RF4 | Subida de archivos de tamaño arbitrario (hasta 65 536 bloques), divididos en bloques de 16 MiB y replicados dos veces. Si ya existe un archivo con el mismo nombre, se le asigna uno nuevo (`a (1).pdf`). |
| RF5 | Publicación atómica de cada subida: el archivo solo se vuelve visible cuando todos sus bloques tienen sus réplicas confirmadas. Reemplazar un archivo crea una nueva versión, y los lectores ven la versión anterior completa o la nueva completa, nunca una mezcla. Una subida fallida se aborta sin dejar rastro visible. |
| RF6 | Descarga y previsualización de archivos desde el navegador, con verificación de integridad de cada bloque y conmutación automática a otra réplica si una falla o está corrupta. |
| RF7 | Re-replicación automática: cuando un bloque queda con menos de dos copias (worker caído, copia corrupta), el líder ordena copiarlo a otro worker, preferiblemente en una AZ que no tenga copia. |
| RF8 | Detección de corrupción: los workers verifican periódicamente todos sus bloques (*scrubbing*), ponen en cuarentena los corruptos y los reportan al master. |

| ID | Requerimiento |
|---|---|
| **Plano de control (master)** | |
| RF9 | Alta disponibilidad del master. Dos masters comparten la metadata en DynamoDB y compiten por un *lease* de líder (10 s, renovado cada 3 s). Solo el líder responde 200 en `/health/leader`, y el balanceador interno le envía todo el tráfico de gateways y workers. Si el líder cae, el otro toma el lease en a lo sumo 10 s y el sistema vuelve a atender en menos de 30 s, sin perder archivos confirmados. Un líder obsoleto no puede escribir metadata ni dar órdenes a los workers (RNF4). |
| RF10 | Monitoreo de workers. Cada worker envía un *heartbeat* cada 3 s con su uso, su cuota y los bloques nuevos y borrados desde el anterior, y un *block report* completo al registrarse, cada 5 min, cuando cambia el líder o cuando el líder se lo pide. A los 10 s sin heartbeat el master lo marca SUSPECT: deja de asignarle bloques nuevos, pero sigue sirviendo lecturas. A los 30 s lo marca DEAD y sus bloques se re-replican (RF7). El mapa de ubicaciones vive solo en la memoria del líder y se reconstruye con los block reports; tras un *failover*, el líder pasa 30 s en modo seguro, sin re-replicar ni borrar, mientras lo reconstruye. |
| **Almacenamiento (worker)** | |
| RF11 | Almacenamiento de bloques por identificador. Cada worker guarda un bloque como un archivo cuyo nombre es su UUID, repartido en dos niveles de carpetas derivados de ese identificador. El worker no conoce el usuario, el archivo ni la ruta lógica a la que pertenece un bloque. Todo identificador que no sea un UUID válido se rechaza antes de tocar el disco, lo que impide escapar de la carpeta de datos (*path traversal*). |
| RF12 | Bloques inmutables. Un bloque almacenado nunca se modifica. Escribir un bloque cuyo identificador ya existe se rechaza con `ALREADY_EXISTS`, salvo en una reparación explícita (RF18). |
| RF13 | Escritura segura en *streaming*. El worker recibe cada bloque en *frames* de tamaño configurable (256 KiB por defecto) y los escribe en un archivo temporal. Solo si el tamaño y el SHA-256 coinciden con los de la cabecera, guarda el hash junto al bloque, sincroniza a disco y publica el bloque con un renombrado atómico, de modo que un bloque visible siempre tiene su hash. Un bloque incompleto, excedido o con hash distinto nunca es visible y no deja archivos temporales. |
| RF14 | Verificación antes de entregar. Antes de enviar un bloque, el worker recalcula su SHA-256 y lo compara con el guardado, sin consultar al master. Si no coincide, o si falta el hash, responde `DATA_LOSS` sin enviar ningún byte, y el gateway pasa de inmediato a otra réplica (RF6). |
| RF15 | Lectura parcial y con contrapresión. El worker entrega un bloque en *frames* a partir de un desplazamiento (*offset*) dado, lo que permite leer rangos y reanudar descargas. Un offset igual al tamaño produce una lectura vacía y uno mayor se rechaza con `OUT_OF_RANGE`. La lectura nunca carga el bloque completo en memoria: si el receptor es lento, la lectura del disco se pausa. |

| ID | Requerimiento |
|---|---|
| RF16 | Cuota de almacenamiento por worker. Cada worker tiene una cuota configurable. El espacio se reserva de forma atómica al iniciar una escritura y, si no cabe, se rechaza con `RESOURCE_EXHAUSTED` antes de recibir datos. La reserva se devuelve si la escritura falla o se abandona, y el espacio se libera al borrar el bloque. |
| RF17 | Borrado de bloques por orden del líder. El worker borra el bloque y su hash, y libera el espacio. La operación es idempotente: borrar un bloque que ya no existe responde con éxito para que el líder pueda reintentar. Un bloque corrupto también se puede borrar. |
| RF18 | Reparación de réplicas por sobrescritura. Cuando el líder lo indica expresamente, el worker reemplaza una copia dañada con una copia verificada. El reemplazo es atómico y, si falla, la copia anterior se conserva con su espacio contabilizado. |
| RF19 | Copia de bloques entre workers por orden del líder. El worker verifica su copia local y la envía a otro worker, que la valida como cualquier escritura. Nunca propaga una copia corrupta. Los errores del destino (bloque ya existente, cuota llena, destino inalcanzable) llegan intactos a quien dio la orden. |
| RF20 | Protección contra líderes obsoletos (*fencing*). Cada orden del líder (borrar, copiar) lleva un número de *fencing*. El worker recuerda el más alto que ha visto y rechaza con `FAILED_PRECONDITION` las órdenes con un número menor, de modo que un master que perdió el liderazgo no puede alterar los datos. |
| RF21 | *Pipeline* de escritura. Si la cabecera indica un siguiente worker, el worker reenvía el bloque mientras lo recibe y solo publica su copia después de que el siguiente confirmó la suya. Si el siguiente falla, no queda ninguna copia en ninguno de los dos y la escritura puede reintentarse con el mismo identificador. |
| RF22 | Arranque consistente. Al iniciar, el worker elimina los temporales huérfanos de ejecuciones interrumpidas, quita las carpetas vacías y recalcula el espacio usado a partir de los bloques que realmente hay en disco. |
| RF23 | Inventario del worker. El worker puede listar todos sus bloques con su tamaño y su hash, sin incluir escrituras en curso. Ese inventario alimenta el heartbeat y el block report (RF10), con los que el líder reconstruye las ubicaciones de las réplicas. |

### Requerimientos no funcionales

Las cifras de rendimiento y latencia son objetivos de diseño calculados a partir de las instancias desplegadas (workers `t3.micro` con disco gp3 de 10 GB; masters y gateways `t3.small`); su verificación está en la Sección 8.

| ID | Requerimiento |
|---|---|
| RNF1 | **Disponibilidad del servicio**: objetivo de 99,9 % de peticiones atendidas al mes (a lo sumo 43,8 min sin servicio), medido como fracción de peticiones exitosas durante las pruebas de falla. Ningún nodo EC2 es punto único de falla (2 masters, 2 gateways y workers con R = 2 en 2 AZ). Las dependencias en serie sin repuesto propio son DynamoDB y los dos NLB, con un SLA de 99,99 % cada uno; con los pares de instancias redundantes (SLA de 99,5 % por instancia) el techo teórico queda en $\approx$ 99,96 %. Cada caída del líder cuesta menos de 30 s, así que el margen admite decenas de *failovers* al mes. Las pausas del Learner Lab, que apaga el entorno por diseño, no cuentan. |
| RNF2 | **Alta disponibilidad del plano de control**: dos masters en AZ distintas en modo activo–pasivo. Ante la caída del líder, el standby asume en a lo sumo $\sim$10 s (duración del *lease*). En pruebas locales el *failover* tomó alrededor de 9 s. |
| RNF3 | **Tolerancia a fallas de datos**: factor de replicación 2 con una réplica por AZ; el sistema tolera la pérdida de un worker o de una AZ completa sin perder archivos confirmados. |
| RNF4 | **Consistencia ante *split-brain***: toda escritura de metadata está protegida por un *fencing token* verificado transaccionalmente en DynamoDB, y los workers rechazan órdenes con un token inferior al más alto que han visto (RF20). |
| RNF5 | **Integridad**: SHA-256 por bloque, verificado por el worker al escribir y antes de entregar (RF13, RF14), por el gateway al leer y periódicamente en reposo (RF8). |
| RNF6 | **Seguridad**: autenticación con JWT (HS256), contraseñas con argon2id, aislamiento de datos entre usuarios, cifrado en reposo de cada bloque con ChaCha20 (los workers solo almacenan texto cifrado) y TLS en el punto de entrada público. Los nodos internos se ubican en subredes privadas sin acceso SSH. |
| RNF7 | **Escalabilidad**: los workers forman un grupo de autoescalado (3 a 6 instancias) y los nuevos nodos reciben bloques preferentemente gracias a la política de ubicación *AvailableSpace*, sin necesidad de un balanceador. Los gateways no tienen estado y pueden replicarse horizontalmente. |
| RNF8 | **Velocidad de escritura**: los datos van directo del gateway a los workers (el master no es cuello de botella), en *pipeline* entre réplicas y con hasta 4 bloques en vuelo por subida. La red base de un `t3.micro` es de 64 Mbit/s ($\approx$ 7,6 MiB/s) y la de un `t3.small` de 128 Mbit/s ($\approx$ 15 MiB/s); el disco gp3 da 125 MiB/s, así que el límite sostenido es la red. Cada bloque recibe una copia en cada AZ, de modo que el techo es la red de entrada de la AZ con menos workers: $\approx$ 7,6 MiB/s con 3 workers $(2+1)$ y $\approx$ 15 MiB/s con 4 $(2+2)$, donde el gateway pasa a ser el límite. Objetivo: al menos 5 MiB/s sostenidos por cliente con la configuración mínima. |

| ID | Requerimiento |
|---|---|
| RNF9 | **Velocidad de descarga**: el gateway lee un bloque a la vez desde un worker `t3.micro`, así que el techo es ≈ 7,6 MiB/s por descarga; las descargas simultáneas se reparten entre réplicas y el total crece con el número de workers, hasta ≈ 15 MiB/s por gateway. Objetivo: al menos 5 MiB/s sostenidos por cliente. En ambos sentidos, mientras haya créditos de red (ráfagas de hasta 5 Gbit/s) el límite pasa a ser el disco y la CPU. |
| RNF10 | **Latencia** (medida desde el gateway, sin el tramo de internet del cliente): operaciones de metadata (`ls`, `mkdir`, `stat`, `mv`, `rm`) con p95 ≤ 100 ms, pues son una llamada gRPC al líder y pocas operaciones en DynamoDB; primer byte de una descarga con p95 ≤ 3 s para un bloque completo de 16 MiB, porque el gateway verifica el bloque entero antes de entregarlo (16 MiB a 7,6 MiB/s ≈ 2,1 s), y ≤ 300 ms para archivos de hasta 1 MiB. |
| RNF11 | **Despliegue reproducible y operación**: infraestructura como código con Terraform, imágenes Docker publicadas por CI y un entorno local completo con Docker Compose. Los servicios se configuran por argumentos, variables de entorno o archivo `.env` (en el worker: identificador, zona, puerto, carpeta de datos, cuota y tamaño de *frame*). La imagen del worker corre sin privilegios de administrador y, al recibir `SIGTERM` o Ctrl+C, deja terminar las peticiones en curso antes de cerrar. |

# 3 Arquitectura del sistema y diagramas

## 3.1 Estilo y decisiones arquitectónicas

Clawsha sigue un estilo cliente–servidor organizado en capas, con un plano de control (master) separado del plano de datos (workers), a la manera de HDFS. El sistema está desplegado en AWS dentro de una VPC (10.0.0.0/16) distribuida en dos zonas de disponibilidad. Cada componente tiene una réplica en ambas zonas, de modo que si una zona falla, el sistema sigue funcionando con la otra.

Las decisiones que dan forma a la arquitectura son:

- **Cuatro capas en subredes separadas**: pública, de acceso (frontend y gateway), de control (master) y de datos (workers). Cada capa solo habla con la siguiente.
- **Ningún nodo expuesto**: ninguna instancia de frontend, gateway, master o worker es accesible directamente desde Internet. Cada grupo de instancias está protegido por su propio *Security Group*, que solo permite el tráfico necesario entre capas.
- **Réplica de cada componente en ambas zonas**: dos nodos de acceso, dos masters (activo–pasivo) y workers en las dos zonas, de modo que los bloques se replican entre ellas.
- **Grupos de autoescalado (ASG)**: todos los nodos pertenecen a un ASG, que reemplaza una instancia caída. El de los workers además crece o se reduce según la carga.

### 3.2 Componentes del sistema

**Capa pública.** En cada zona hay una subred pública (10.0.0.0/24 y 10.0.1.0/24) conectada a Internet mediante un *Internet Gateway*. En estas subredes se encuentran el *Network Load Balancer* (NLB) público, que recibe las peticiones de los clientes, y los *NAT Gateways*, que permiten a las instancias privadas salir a Internet sin quedar expuestas.

**Capa de acceso (frontend y gateway).** En las primeras subredes privadas están el frontend y el gateway, que comparten nodo: un contenedor de nginx sirve el frontend y pasa las peticiones `/api/` al gateway. El NLB público reparte el tráfico entre estos nodos, uno por zona, que pertenecen a un ASG encargado de reponer el que falle; como el gateway no guarda estado, se pueden agregar más nodos sin coordinación. El gateway es el punto de entrada al sistema: recibe las solicitudes y envía las de metadatos a un NLB interno.

**Capa de control (master).** Los masters están en un segundo nivel de subredes privadas, uno en cada zona. El NLB interno envía todas las solicitudes al master líder, el único que responde 200 en `/health/leader`; el otro espera como *standby* (RF9). El master gestiona los metadatos del sistema de archivos, es decir, qué archivos existen (guardados en DynamoDB) y en qué workers están sus bloques (en memoria, RF10). Luego responde al gateway con esa información.

**Capa de datos (workers).** Con la información del master, el gateway se comunica directamente con los workers, que están en las últimas subredes privadas y forman parte de un ASG de 3 a 6 instancias. Los workers almacenan los bloques de los archivos. Como hay workers en ambas zonas, cada bloque se replica entre ellas.

### 3.3 Diagramas de arquitectura y despliegue

La Figura 1 muestra la VPC con sus dos zonas de disponibilidad, las cuatro capas de subredes, los dos NLB y los grupos de autoescalado.

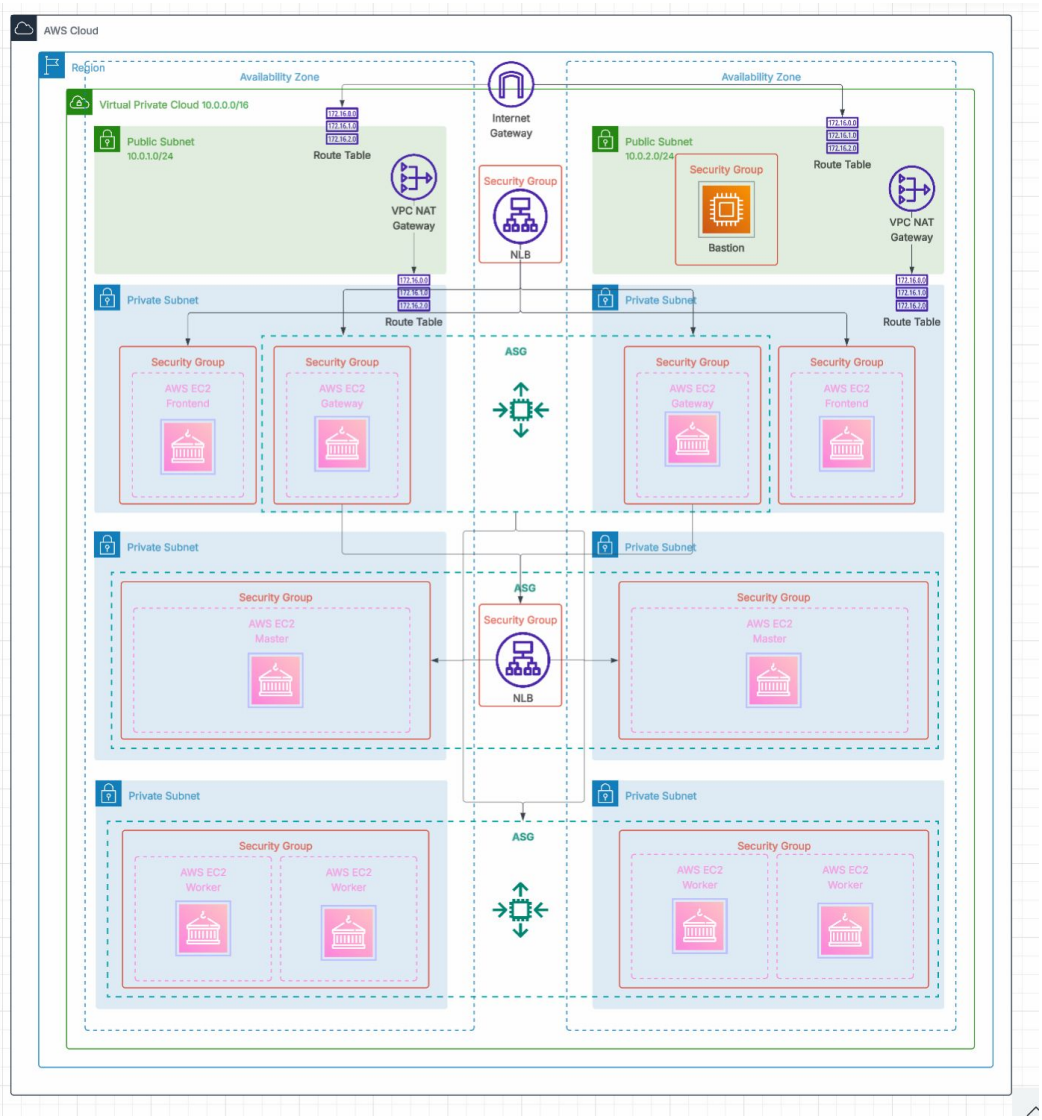



Figura 1: Arquitectura del sistema Clawsha

## 3.4 Flujos principales de operación

Toda petición de un cliente recorre las capas en el mismo orden; los datos de los archivos nunca pasan por el master:

1. El navegador envía la petición al NLB público, que la entrega al nginx de un nodo de acceso, y este la pasa al gateway.
2. El gateway valida el JWT y consulta al master líder a través del NLB interno: crear una subida, obtener un plan de lectura o modificar el espacio de nombres.
3. El master responde con los metadatos: los bloques del archivo y los workers donde están (o donde deben quedar) sus réplicas.
4. Con esa información, el gateway lee o escribe los bloques directamente en los workers por gRPC, y en una escritura los workers se reenvían el bloque en *pipeline*.

Los flujos detallados de subida y descarga, con cada llamada gRPC, están en la Sección 4.

### Diagrama de comunicación general del sistema

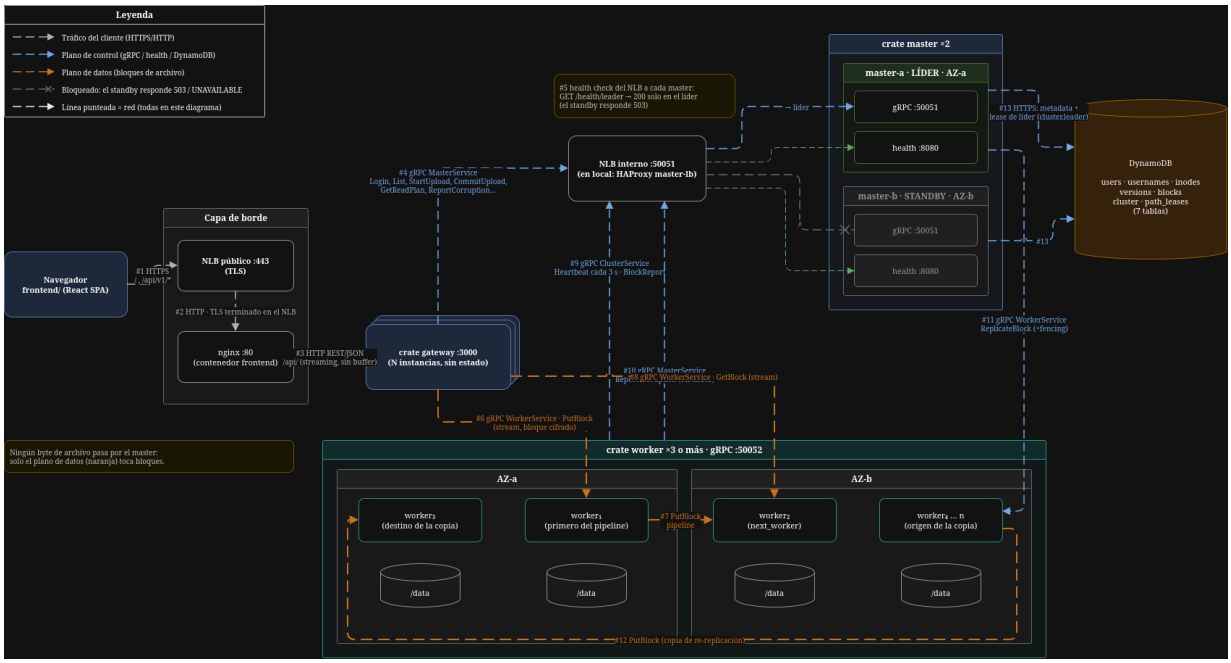



Figura 2: Arquitectura de comunicación general

### Diagrama de comunicación del master

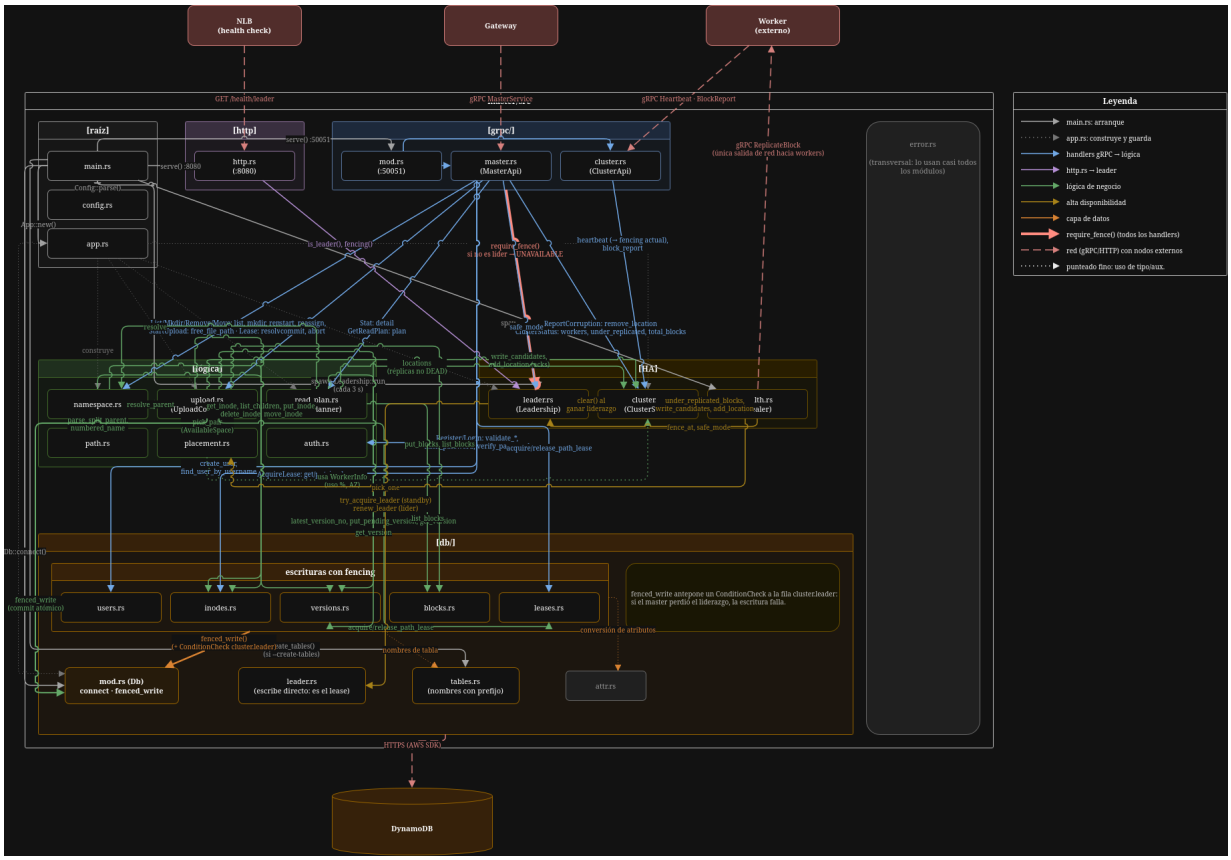



Figura 3: Arquitectura de comunicación del master

## Diagrama de comunicación del worker

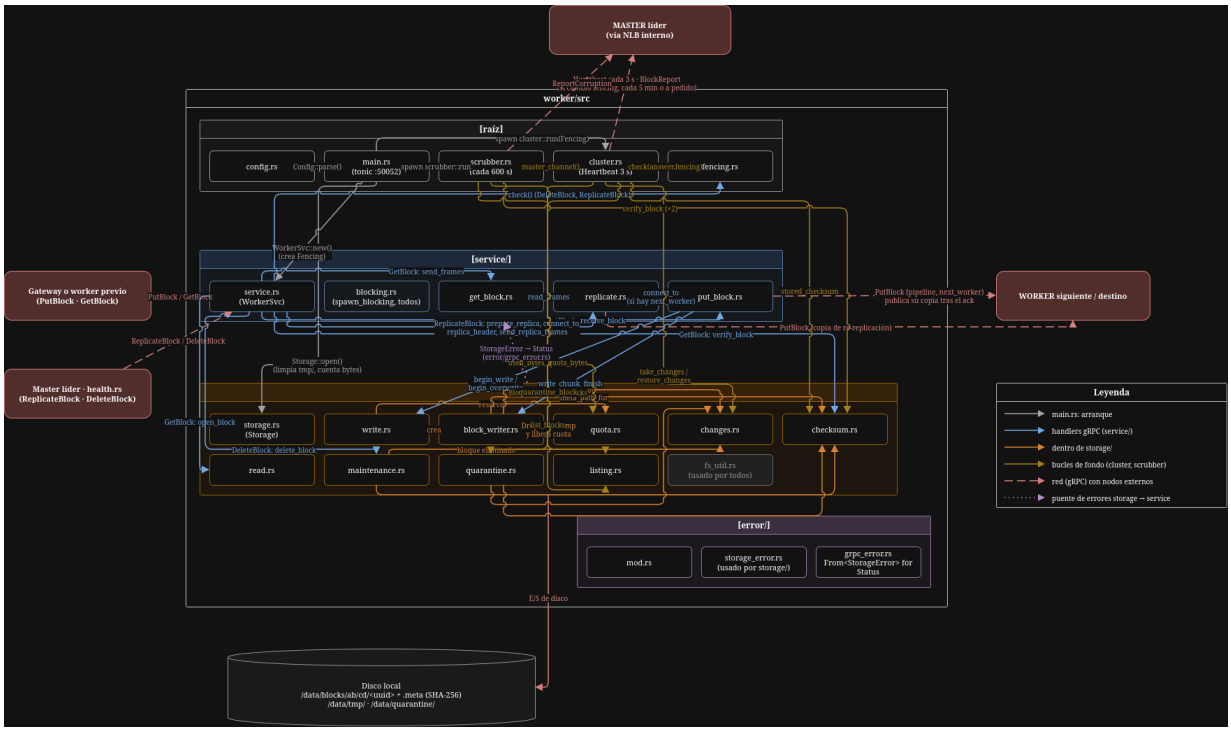



Figura 4: Arquitectura de comunicación del worker

### Diagrama de comunicación del gateway

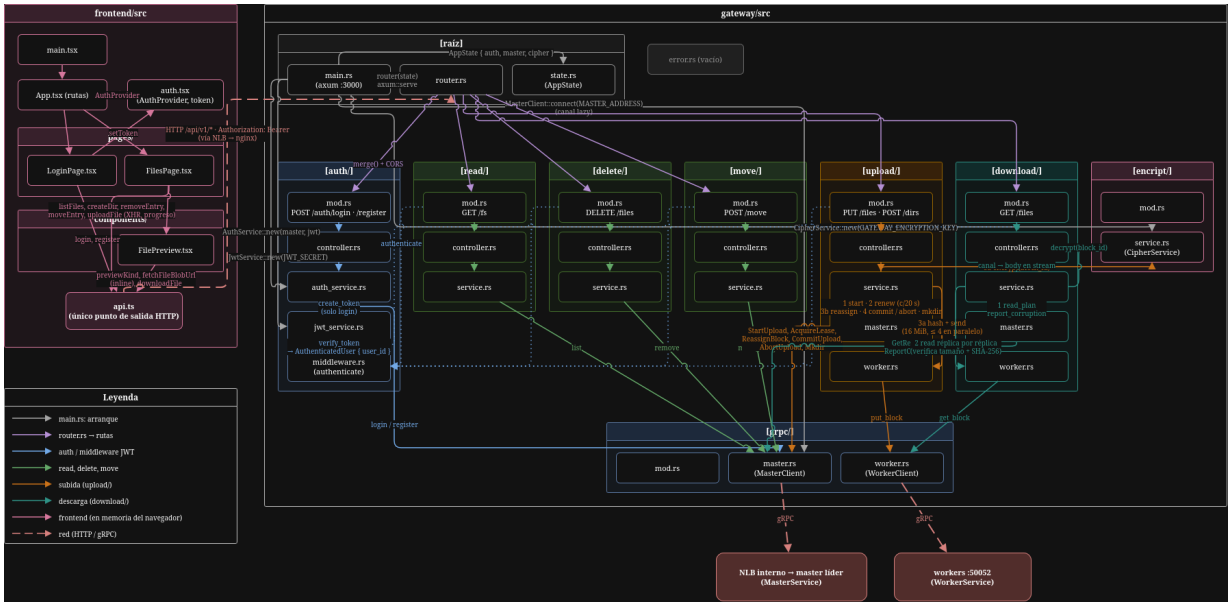



Figura 5: Arquitectura de comunicación del gateway

# 4 Especificaciones del protocolo y APIs

## 4.1 Protocolos de comunicación

DFSha utiliza dos protocolos de aplicación: **HTTP/1.1 con JSON** (REST) entre el navegador y el sistema, y **gRPC** (HTTP/2 con Protocol Buffers v3) para la comunicación entre los componentes internos. El contrato gRPC se encuentra en `proto/` (paquete `dfs.v1`) y se compila en cada *crate* mediante `tonic-prost-build`. De esta manera, todos los componentes comparten las mismas definiciones de mensajes.

| Tramo | Protocolo | Puerto | Observaciones |
|---|---|---|---|
| Navegador → NLB público | HTTPS (TLS 1.2/1.3) | 443 | El NLB termina TLS. Utiliza la política `ELBSecurityPolicy-TLS13-1-2-2021-06` y ALPN HTTP/1.1. |
| NLB → nginx (frontend) | HTTP/1.1 | 80 | Sirve la SPA de React y reenvía `/api/` al gateway sin *buffering* y sin límite de tamaño del cuerpo. `/healthz` se utiliza como chequeo del NLB. |
| nginx → gateway | HTTP/1.1, REST + JSON | 3000 | API disponible bajo `/api/v1/*`. |
| Gateway → master | gRPC (`MasterService`) | 50051 | La comunicación pasa por el NLB interno, que solo enruta al líder. |
| Worker → master | gRPC (`ClusterService`) | 50051 | Envía un *heartbeat* cada 3 s y un *block report* al arrancar, cada 5 min y cuando cambia el *fencing token*. |
| Gateway / master / worker → worker | gRPC (`WorkerService`) | 50052 | Transfiere bloques mediante *streaming*, en tramas de 256 KiB (máximo de 1 MiB). |
| NLB interno → master | HTTP/1.1 | 8080 | `/health` indica que el proceso está activo y `/health/leader` devuelve 200 solo en el líder y 503 en el nodo en espera (*standby*). |
| Master → DynamoDB | HTTPS (API de AWS) | 443 | La comunicación utiliza el *VPC endpoint* de DynamoDB, sin salir a Internet. |

**Transferencia de bloques.** Un bloque se transmite como un flujo (*stream*) de mensajes `BlockFrame`. La primera trama contiene un `BlockHeader` con `block_id`, el SHA-256 esperado, el tamaño, la dirección del siguiente *worker* y la bandera `overwrite`. Las tramas siguientes son mensajes `DataFrame` que contienen los datos. Durante una escritura, el gateway envía el bloque al primer worker, que guarda cada trama y la reenvía al segundo mediante un esquema de *pipeline*. El primer worker confirma la operación cuando el segundo ha confirmado la persistencia; por tanto, un *ack* del primer worker indica que existen dos copias persistidas.

**Descubrimiento del líder.** Los clientes del master (gateways y workers) no necesitan conocer cuál nodo master es el líder: se conectan al DNS del NLB interno. En el entorno local, se utiliza HAProxy en `master-lb:50051`. El *health check* sobre `/health/leader` excluye al nodo *standby* del enrutamiento. Si una petición llega a un nodo que no es líder, este responde con `UNAVAILABLE`.

## 4.2 Especificación de las APIs

### API REST pública (gateway)

Todas las rutas están bajo `/api/v1`. Excepto las rutas de `auth/*`, las solicitudes requieren la cabecera `Authorization: Bearer <JWT>`. Si el token no se envía, es inválido o está vencido, la respuesta es `401`. Las rutas de archivos son absolutas dentro del árbol del usuario (por ejemplo, `/docs/a.pdf`). Los errores se devuelven únicamente mediante el código HTTP, sin cuerpo.

| Método y ruta | Parámetros | Respuesta exitosa | Errores |
|---|---|---|---|
| `POST /auth/register` | JSON `{username, password}`. Usuario de 3–32 caracteres, con patrón `[A-Za-z0-9._-]`; contraseña de 8–128 caracteres. | `201: {user_id}` | 400, 409, 503 |
| `POST /auth/login` | JSON `{username, password}` | `200: {user_id, token}` | 400, 401, 503 |
| `GET /fs` | `?path=` (por defecto, `/`) | `200`: listado con `kind`, `size` y `modified_at`. `kind` puede ser `directory` o `file`; `modified_at` se expresa en milisegundos Unix. | 400, 404, 503 |
| `POST /dirs` | `?path=`. No es recursivo: el directorio padre debe existir. | Código de éxito no especificado en el texto fuente. | 400, 404, 409, 503 |
| `PUT /files` | `?path=`; cuerpo con los bytes sin procesar del archivo. `Content-Length` es obligatorio. | `201: {path, inode_id, version_no, size, sha256, block_count, modified_at}`. Si el nombre está ocupado, `path` puede cambiar, por ejemplo, a `a (1).pdf`. | 400, 404, 409, 411, 502, 503 |
| `GET /files` | `?path=` y `?disposition=attachment\|inline` | `200`: contenido en *streaming*, con `Content-Type` según la extensión, `Content-Length` y `Content-Disposition`. | 400, 404, 502, 503 |
| `DELETE /files` | `?path=` y `?recursive=true\|false` (por defecto, `false`) | `204` | 400, 404, 409, 503 |
| `POST /move` | JSON `{from, to}`; `to` es la ruta de destino completa, incluido el nombre. | `204` | 400, 404, 409, 503 |

**Semántica de los códigos de error.**

| Estado gRPC | HTTP | Causa |
|---|---|---|
| `INVALID_ARGUMENT` | 400 | Ruta relativa; componentes `.` o `..`; nombre de más de 255 bytes o con caracteres de control; credenciales que no cumplen las reglas; o reporte de *commit* inconsistente. |
| `UNAUTHENTICATED` | 401 | Usuario o contraseña incorrectos. |
| `NOT_FOUND` | 404 | La ruta no existe o pertenece a otro usuario. |
| `ALREADY_EXISTS` | 409 | Nombre de usuario o directorio ya ocupado; destino de `move` ocupado. |
| `FAILED_PRECONDITION` | 409 | Directorio no vacío cuando se intenta eliminarlo sin una operación recursiva. |
| No aplica | 411 | Subida sin la cabecera `Content-Length`. |

| Estado gRPC | HTTP | Causa |
|---|---|---|
| `UNAVAILABLE` | 503 | El master consultado no es el líder; no se encuentra un worker con espacio disponible; algún bloque no tiene una réplica activa; o falla la renovación del *lease* de subida. |
| Otros errores | 502 | Fallo inesperado de un worker o del master. |

**Flujo interno de una subida.** La solicitud `PUT /files` se traduce en las siguientes operaciones: (1) `StartUpload`, que crea una versión `PENDING`, adquiere el *lease* de la ruta y devuelve un plan con un `block_id` y un par de workers por bloque; (2) para cada bloque de 16 MiB, cifrado y envío mediante `PutBlock` en *pipeline*, con hasta cuatro bloques en paralelo; si falla un par de workers, se ejecuta `ReassignBlock` con un identificador y un par nuevos, y se reintenta la operación; (3) renovación del *lease* cada 20 s mediante `AcquireLease`; y (4) `CommitUpload`, que registra el SHA-256 del archivo y los workers que confirmaron cada bloque, o `AbortUpload` si ocurre un error.

**Flujo interno de una descarga.** La solicitud `GET /files` invoca `GetReadPlan`. Para cada bloque, en orden, el gateway lo lee por completo desde una réplica mediante `GetBlock`, verifica su tamaño y SHA-256, lo descifra y lo envía al cliente. Si una réplica falla o está corrupta, el gateway intenta utilizar la siguiente. Solo reporta la réplica al master mediante `ReportCorruption` cuando el hash que él calcula no coincide; si es el worker quien detecta el daño y responde `DATA_LOSS`, el gateway cambia de réplica sin reportarla, y la reparación queda a cargo del *scrubber* (RF8). Cada bloque se verifica antes de enviar sus bytes; si ninguna réplica funciona, la descarga se interrumpe.

## APIs internas (gRPC)

| Servicio | RPC | Propósito |
|---|---|---|
| `MasterService` (gateway → master) | `Register`, `Login` | Registro y verificación de usuarios; devuelven el `user_id`. |
| | `List`, `Mkdir`, `Remove`, `Move`, `Stat` | Gestión del espacio de nombres del usuario. `Stat` incluye versión, hash y bloques con sus réplicas. |
| | `StartUpload`, `ReassignBlock`, `CommitUpload`, `AbortUpload` | Gestión del ciclo de vida de una subida. |
| | `GetReadPlan`, `ReportCorruption` | Plan de lectura y reporte de réplicas corruptas. |
| | `AcquireLease`, `ReleaseLease` | Bloqueo de escritura por ruta (TTL de 60 s por defecto). |
| | `ClusterStatus` | Estado del líder, *fencing*, modo seguro, estado y uso de cada worker, y bloques con menos réplicas de las previstas. |
| `ClusterService` (worker → master) | `Heartbeat` | Uso, cuota y bloques nuevos o eliminados desde el último latido. La respuesta incluye el *fencing token* vigente e indica si el líder solicita un reporte completo. |
| | `BlockReport` | Lista completa de bloques almacenados en disco; se utiliza como fuente de verdad para localizar los bloques. |
| `WorkerService` | `PutBlock` (*client streaming*) | Recibe un bloque y, opcionalmente, lo reenvía al siguiente worker. |

| Servicio | RPC | Propósito |
|---|---|---|
| | `GetBlock` (*server streaming*) | Verifica el bloque mediante su hash y lo envía desde un *offset*. |
| | `ReplicateBlock` | Orden del líder para copiar un bloque a otro worker. |
| | `DeleteBlock` | Orden del líder para eliminar un bloque. |

Las órdenes `ReplicateBlock` y `DeleteBlock` incluyen el *fencing token* del líder que las emite. El worker las rechaza con `FAILED_PRECONDITION` si el token es inferior al valor más alto que ha observado.

Todo fallo de almacenamiento del worker se traduce a un único código gRPC, y el gateway decide qué hacer según el código: `INVALID_ARGUMENT` (identificador que no es un UUID), `NOT_FOUND`, `ALREADY_EXISTS` (bloque ya escrito), `DATA_LOSS` (hash distinto o ausente: cambia de réplica), `OUT_OF_RANGE` (*offset* mayor que el tamaño), `RESOURCE_EXHAUSTED` (cuota llena) e `INTERNAL`.

## 4.3 Autenticación, autorización y seguridad

**Autenticación de usuarios.** Las contraseñas se almacenan con **Argon2id**, utilizando una sal aleatoria por usuario, en la tabla `users` de DynamoDB. El registro escribe en `users` y `usernames` mediante una única transacción con condición de no existencia, para impedir que dos usuarios tengan el mismo nombre. Tras un inicio de sesión correcto, el gateway emite un **JWT firmado con HS256**, cuyo campo `sub` contiene el `user_id`. El token tiene una vigencia de una hora y no admite tolerancia en la expiración. En AWS, Terraform genera el secreto de firma (48 caracteres aleatorios), que comparten todos los gateways; estos no almacenan estado de sesión.

**Autorización y aislamiento.** Un *middleware* de Axum valida el JWT en todas las rutas protegidas y extrae el `user_id`; el cliente nunca lo envía directamente. El master recibe ese identificador en cada petición y resuelve las rutas desde la raíz virtual del usuario (`root:<user_id>`). Así, el árbol de otro usuario no es accesible y cualquier intento de acceder a él devuelve `404`, sin revelar si la ruta existe. Además, `CommitUpload` comprueba que la versión pendiente pertenezca al mismo usuario.

**Confidencialidad e integridad de los datos.**

- **Cifrado en reposo:** el gateway cifra cada bloque con **ChaCha20** (clave de 256 bits) antes de enviarlo a los workers, que almacenan únicamente el texto cifrado. Se eligió un cifrador de flujo porque conserva la longitud y respeta los tamaños que valida el master. El *nonce* se deriva del `block_id`, un UUID v4 nuevo en cada asignación y reasignación, con el objetivo de evitar la reutilización de un mismo par (clave, *nonce*). Además, el volumen EBS de cada worker está cifrado.
- **Integridad:** el SHA-256 del texto cifrado de cada bloque se verifica en el worker durante la escritura y la lectura, en el gateway durante la descarga y periódicamente mediante el *scrubber*. El master también almacena el SHA-256 del archivo completo, calculado sobre el texto plano.
- **Cifrado en tránsito:** se utiliza TLS 1.2/1.3 entre el navegador y el NLB público, con un certificado autofirmado generado por Terraform e importado en ACM.

**Protección del plano de control.** Para evitar el *split-brain*, cada escritura del master en DynamoDB se realiza mediante una transacción cuyo primer elemento verifica que el *lease* de líder siga asociado al nodo que realiza la operación y a su *fencing token*. Si otro master asumió el liderazgo, la transacción completa se rechaza. Los workers aplican la misma regla a las órdenes que reciben.

**Protección de la web.** Las descargas con `disposition=inline` solo se permiten para tipos considerados seguros (imágenes excepto SVG, PDF, texto plano, audio y video). Los archivos HTML y SVG se fuerzan a descarga para evitar que un archivo subido ejecute scripts en el origen de la aplicación. También se envía la cabecera `X-Content-Type-Options: nosniff`, se sanea el nombre del archivo en `Content-Disposition` y CORS solo admite los orígenes configurados en `CORS_ALLOWED_ORIGINS`.

**Seguridad de red (AWS).** Los masters, workers y gateways se ejecutan en subredes privadas. Desde Internet solo es accesible el NLB público. Cada rol tiene su propio *security group*; los puertos del master (50051 y 8080) y del worker (50052) solo aceptan tráfico desde la VPC. No se expone el puerto 22: el acceso administrativo se realiza mediante SSM Session Manager. Todas las instancias requieren IMDSv2 y las tablas de DynamoDB tienen habilitada la recuperación a un punto en el tiempo (*point-in-time recovery*).

# 5 Algoritmo de particionamiento y distribución

Clawsha divide cada archivo en bloques de tamaño fijo y reparte cada bloque, de forma independiente, en dos workers de zonas de disponibilidad distintas. El master decide la partición y la ubicación; el gateway corta los bytes y los envía. Ningún byte de datos pasa por el master.

## 5.1 Estrategia de particionamiento

**Bloques de tamaño fijo.** Un archivo de $S$ bytes se divide en

$$n = \left\lceil \frac{S}{B} \right\rceil \text{ bloques}, \qquad \text{tamaño}(i) = \begin{cases} B & 0 \le i < n-1, \\ S - (n-1)\,B & i = n-1, \end{cases}$$

donde $B$ es el tamaño de bloque, 16 MiB (16 777 216 bytes) por defecto y configurable con `MASTER_BLOCK_SIZE`. Todos los bloques están llenos excepto, posiblemente, el último. Un archivo vacío tiene cero bloques y solo existe como metadata. El tamaño $S$ llega del `Content-Length` de la petición, así que el master rechaza los planes de más de 65 536 bloques (1 TiB con bloques de 16 MiB) antes de reservar nada. La Tabla 7 muestra la partición de algunos de los archivos usados en las pruebas.

**Por qué 16 MiB.** HDFS usa 128 MiB y GFS 64 MiB porque están pensados para archivos de terabytes. En Clawsha el bloque es más pequeño por cuatro razones:

- **Distribución real con archivos medianos.** Con 16 MiB, un archivo de 100 MiB ya ocupa 7 bloques repartidos entre varios workers. Con 128 MiB, cabría en un solo bloque y en un solo par de workers.

Tabla 7: Partición de archivos de prueba con $B = 16$ MiB

| Archivo | Tamaño | Bloques | Tamaños de bloque |
|---|---|---|---|
| vacío | 0 B | 0 | – |
| un byte | 1 B | 1 | 1 B |
| un bloque exacto | 16 MiB | 1 | 16 MiB |
| un bloque + 1 B | 16 MiB + 1 B | 2 | 16 MiB, 1 B |
| mediano | 100 MiB | 7 | $6 \times 16$ MiB, 4 MiB |
| grande | 1 GiB | 64 | $64 \times 16$ MiB |

- **Memoria del gateway.** El gateway guarda en memoria cada bloque completo para cifrarlo y calcular su hash, y en la lectura para verificarlo antes de entregarlo. Con 4 bloques en vuelo por subida son 64 MiB por cliente; con bloques de 64 MiB serían 256 MiB, demasiado para un `t3.small` con varios clientes.
- **Latencia del primer byte.** Como el gateway verifica el bloque entero antes de enviar su primer byte, el primer byte de una descarga tarda lo que tarda en llegar un bloque completo: unos 2 s a 7,6 MiB/s (RNF10).
- **Metadata acotada.** Cada bloque es una fila en DynamoDB y una entrada en el mapa de ubicaciones en memoria del líder; 16 MiB mantiene ambos pequeños para los volúmenes del proyecto.

Dentro de un bloque, los datos viajan en *frames* de 256 KiB para no acercarse al límite de 4 MiB por mensaje de gRPC. El efecto medido del tamaño de bloque está en la Sección 8.3.

**Identificación de los bloques.** Cada bloque se identifica por un UUID v4 que el master genera al asignarlo, no por un hash de su contenido. La tabla `blocks` de DynamoDB relaciona la versión del archivo y la posición del bloque con su identificador:

$$(\texttt{version_id},\ \texttt{idx}) \longmapsto (\texttt{block_id},\ \text{tamaño},\ \text{SHA-256}),$$

donde `version_id` combina el *inode* del archivo y el número de versión. Así, el worker no sabe a qué usuario, archivo o ruta pertenece un bloque: solo guarda `blocks/ab/cd/<uuid>` con su `.meta` (RF11). Hay tres razones para usar un identificador aleatorio:

- El *nonce* de ChaCha20 se deriva del `block_id`. Un identificador nuevo en cada asignación garantiza que nunca se repite un par (clave, *nonce*), aunque se suba dos veces el mismo contenido.
- Los bloques son inmutables (RF12). Si una escritura falla y se reasigna, el bloque recibe un UUID nuevo, de modo que una copia a medias con el identificador viejo nunca se confunde con la buena.
- El reparto en carpetas `ab/cd` es uniforme porque los UUID son aleatorios.

El hash que guarda la tabla `blocks` es el SHA-256 del texto cifrado (el que almacenan y verifican los workers), mientras que la versión del archivo guarda el SHA-256 del archivo completo en texto plano.

**Corte en el gateway.** El gateway recibe el cuerpo HTTP como un flujo y, para cada bloque del plan, acumula exactamente tamaño($i$) bytes. Con ellos actualiza el hash del archivo completo, cifra el bloque con ChaCha20 usando su `block_id`, calcula el SHA-256 del texto cifrado y lo envía con `PutBlock` al primer worker de su par, que lo reenvía al segundo en *pipeline* (RF21). Hay como máximo 4 bloques en vuelo por subida. Si sobran o faltan bytes respecto al tamaño anunciado, la subida se aborta.

## 5.2 Algoritmo de distribución

El master asigna los bloques uno por uno en `StartUpload`, siguiendo la Figura 6. Tres reglas guían la asignación:

1. **Solo workers sanos y con espacio.** Son candidatos los workers en estado ALIVE (no SUSPECT ni DEAD) con espacio libre para al menos dos bloques ($2B$), de modo que una asignación nunca llena un disco hasta el borde.
2. **Una réplica por AZ.** La primera réplica puede ir a cualquier candidato; la segunda se elige entre los candidatos de otra AZ. Así, la caída de una zona completa deja al menos una copia de cada bloque. Si la otra AZ no tiene candidatos (está caída o llena), el master acepta un host distinto de la misma AZ en vez de rechazar la escritura (modo degradado).
3. **Contabilidad local.** Al elegir un par, el master suma el tamaño del bloque al uso de ambos workers en su copia local de la lista. Sin esto, un archivo de 1 GiB iría entero al worker que estaba más vacío al empezar, y un worker que se llena a mitad del plan dejaría de ser candidato demasiado tarde.

**Regla AvailableSpace.** Para elegir un worker dentro de un conjunto de candidatos, Clawsha implementa la política `AvailableSpaceBlockPlacementPolicy` de HDFS con sus valores por defecto (Figura 7): toma dos candidatos al azar y, si su porcentaje de uso difiere en más de 5 puntos, elige el más vacío con probabilidad 0,6 y el más lleno con probabilidad 0,4; si la diferencia es menor, elige cualquiera de los dos con la misma probabilidad.

Se eligió esta regla, en lugar de enviar siempre al worker más vacío o de usar *hashing* consistente, por tres motivos:

- **Los workers nuevos se llenan solos.** Un worker que el ASG agrega llega con 0 % de uso y gana la mayoría de las comparaciones en las que participa, de modo que recibe más bloques nuevos que los demás sin mover los existentes y sin un proceso de rebalanceo. Con *hashing* consistente, en cambio, agregar un nodo obliga a mover datos.
- **Sin puntos calientes.** Elegir siempre el más vacío enviaría todos los bloques de una ráfaga de subidas al mismo worker. Con dos candidatos al azar y una preferencia de 0,6 (no de 1,0), la carga se reparte.
- **El master ya conoce las ubicaciones.** Como el líder guarda un mapa de qué worker tiene cada bloque (necesario porque las reparaciones mueven copias), la ubicación no tiene que poder calcularse a partir del identificador, y se puede decidir con el uso real de cada worker.

El efecto es un sesgo moderado, no una preferencia absoluta: un worker vacío entre $k$ candidatos entra en la comparación con probabilidad $2/k$ y gana con probabilidad 0,6, es decir, recibe la

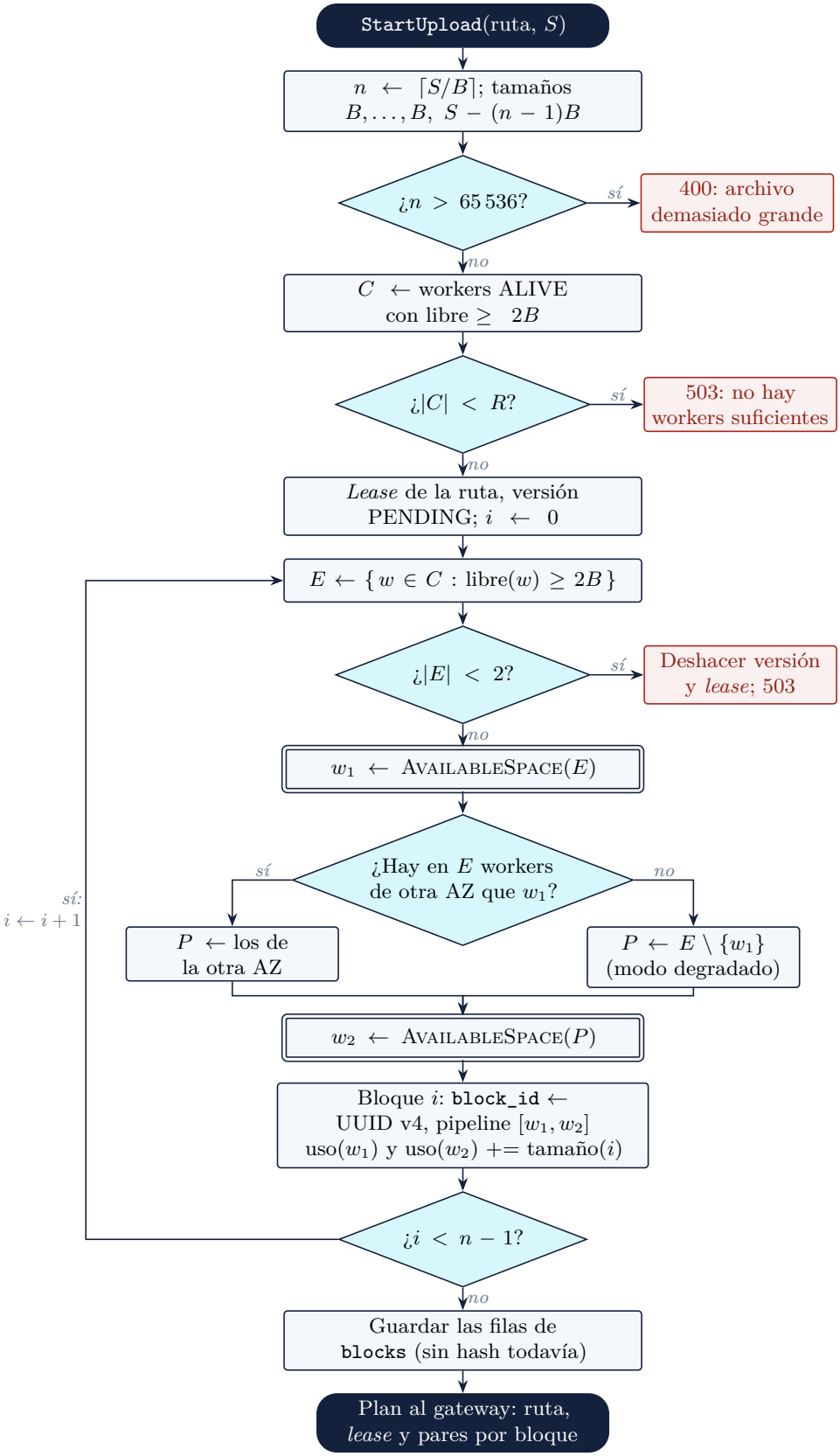



Figura 6: Asignación de bloques en `StartUpload`. Los rectángulos de doble borde invocan la regla AvailableSpace (Figura 7). $B$ es el tamaño de bloque y $R = 2$ el factor de replicación.

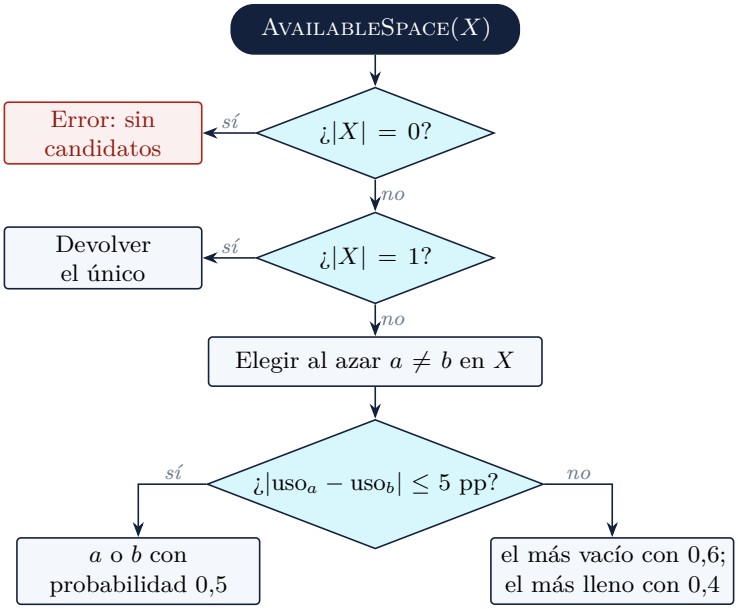



Figura 7: Regla AvailableSpace para elegir un worker entre varios candidatos (`crates/master/src/placement.rs`).

primera réplica con probabilidad $1{,}2/k$, un 20 % más que con una elección uniforme. Para la segunda réplica, el conjunto de candidatos se reduce a la otra AZ y el sesgo pesa más: con dos workers en esa zona, el vacío la recibe el 60 % de las veces. La Sección 8.3 mide este comportamiento al agregar un worker vacío a un clúster cargado.

**Reasignación ante fallas durante la escritura.** Si el envío de un bloque falla (worker caído, cuota llena, *pipeline* roto), el gateway pide `ReassignBlock` excluyendo los dos workers del par. El master vuelve a aplicar el algoritmo anterior para ese único bloque con los candidatos restantes y le asigna un UUID nuevo; el gateway lo vuelve a cifrar con el nuevo *nonce* y lo envía otra vez. Si el segundo intento también falla, la subida se aborta.

## 5.3 Reconstrucción y recuperación de archivos

Para leer un archivo, el gateway pide un plan de lectura y reconstruye el archivo bloque por bloque, en orden:

1. **Plan de lectura (`GetReadPlan`).** El master resuelve la ruta dentro del árbol del usuario, toma la versión confirmada actual (las versiones PENDING nunca son visibles) y lista sus bloques en orden de `idx`. Para cada uno incluye su tamaño, su SHA-256 y los workers no muertos que lo tienen según el mapa en memoria. Si algún bloque no tiene ninguna réplica viva, el master rechaza el plan con 503: es preferible a una descarga que se corte a la mitad.
2. **Lectura con verificación.** El gateway envía la cabecera HTTP con el tamaño total y recorre los bloques. Para cada bloque pide `GetBlock` a la primera réplica; el worker recalcula el hash antes de enviar (RF14) y el gateway vuelve a comprobar el tamaño y el SHA-256 al recibirlo. Solo entonces lo descifra y lo pasa al cliente.

3. **Conmutación de réplica.** Si una réplica no responde, responde `DATA_LOSS` o entrega un hash distinto, el gateway pasa a la siguiente. Cuando es el propio gateway quien detecta el hash distinto, además llama `ReportCorruption`. Si ninguna réplica sirve, la respuesta HTTP se corta con error, de modo que el cliente ve una descarga fallida y nunca datos incorrectos.
4. **Memoria acotada.** Entre el bloque en curso y un canal de dos bloques verificados, el gateway nunca guarda más de tres bloques por descarga, sin importar el tamaño del archivo.

Una vez obtenido el plan, la descarga ya no depende del master: si el líder cae a mitad de una descarga, esta sigue contra los workers. Como el plan apunta a una versión inmutable, un reemplazo simultáneo del archivo no mezcla contenidos: la versión anterior sigue legible durante los 5 min de gracia del recolector de basura.

## 5.4 Replicación, consistencia y tolerancia a fallos

**Escritura replicada y confirmación atómica.** Cada bloque se escribe con $R = 2$ copias en *pipeline*: el primer worker solo confirma cuando el segundo confirmó la suya, así que un *ack* significa dos copias en disco. Al terminar, `CommitUpload` verifica que el número, el orden y los tamaños de los bloques coincidan con la partición, que cada uno tenga un SHA-256 válido y al menos $R$ copias, y publica el archivo en una sola transacción de DynamoDB: la versión pasa a COMMITTED, el *inode* apunta a ella, la versión anterior pasa a OLD y se libera el *lease* de la ruta. La transacción incluye la condición de *fencing* del líder (RNF4), así que un master obsoleto no puede confirmar nada. Mientras dura la subida, el *lease* de 60 s sobre la ruta, renovado cada 20 s, impide que otro cliente escriba en ella (RF3).

**Reacción ante fallas.** La Tabla 8 resume cómo se detecta y se corrige cada tipo de falla. Los tiempos medidos están en la Sección 8.4.

Tabla 8: Detección y reacción ante fallas

| Falla | Detección | Reacción |
|---|---|---|
| Worker caído | Sin *heartbeat*: SUSPECT a los 10 s (deja de recibir bloques nuevos), DEAD a los 30 s. | El ciclo de re-replicación del líder (cada 5 s, tras 10 s de gracia) ordena `ReplicateBlock` desde una copia viva hacia otro worker, de preferencia en la AZ sin copia y elegido con AvailableSpace. Como máximo 2 copias simultáneas por worker. |
| Copia corrupta | El worker verifica el hash al leer (`DATA_LOSS`) y el *scrubber* recorre todos los bloques periódicamente. | El gateway cambia de réplica. El *scrubber* pone la copia en cuarentena y la reporta; el bloque aparece como sub-replicado y se repara como en el caso anterior. |
| Caída de una AZ | Igual que los casos anteriores, para todos sus nodos. | Cada bloque tiene una copia en la otra AZ y el master de esa zona toma el liderazgo. Las reparaciones van a otro host de la misma AZ (modo degradado). |
| Caída del líder | El *lease* de líder (10 s, renovado cada 3 s) expira. | El standby lo toma con un *fencing token* mayor, pasa 30 s en modo seguro (sin reparar ni borrar) y reconstruye el mapa de ubicaciones con los *block reports*. |

| Falla | Detección | Reacción |
|---|---|---|
| Worker que vuelve | Su *block report* anuncia copias que ya se re-replicaron. | El recolector de basura borra la copia sobrante en la AZ con más copias. |
| Subida interrumpida | La versión queda PENDING sin *lease* vigente. | Nunca es visible. Tras 1 h el recolector la marca OLD, a los 5 min borra sus filas y a los 60 s borra los bloques huérfanos. Como freno, no borra nada si los huérfanos que él no produjo son 10 o más y superan la mitad de los bloques (por ejemplo, si la metadata se perdió). |

# 6 Descripción del entorno nativo o Docker

## 6.1 Infraestructura y topología de despliegue

Toda la infraestructura de Clawsha se despliega en AWS (región `us-east-1`, dos zonas de disponibilidad) con Terraform, una herramienta de infraestructura como código. Terraform crea la VPC, las subredes públicas y privadas en las dos zonas, los *NAT Gateways*, los *Security Groups*, los balanceadores (NLB público e interno), los grupos de autoescalado y las instancias EC2, además de las tablas de DynamoDB y el *VPC endpoint* para llegar a ellas sin salir a Internet.

Los tipos de instancia se eligieron según la carga de cada componente:

| Componente | Instancia | Nodos | Subredes |
|---|---|---|---|
| NLB público y NAT | – | 1 NLB, 1 NAT por zona | 10.0.0.0/24, 10.0.1.0/24 (públicas) |
| Frontend (nginx) y gateway | `t3.small` | 2 (uno por zona) | 10.0.30.0/24, 10.0.31.0/24 |
| Master | `t3.small` | 2 (uno por zona) | 10.0.10.0/24, 10.0.11.0/24 |
| Worker | `t3.micro` + gp3 de 10 GB | 3 a 6 (ASG) | 10.0.20.0/24, 10.0.21.0/24 |

- **Gateway y master (`t3.small`)**: el gateway cifra y calcula el SHA-256 de cada byte que entra o sale y verifica contraseñas con argon2id; el master atiende todas las peticiones de metadatos y mantiene en memoria el mapa de ubicaciones de los bloques.
- **Worker (`t3.micro`)**: su trabajo es leer y escribir bloques, limitado por la red y el disco más que por la CPU. La capacidad se obtiene agregando workers al ASG en lugar de agrandar cada uno.

## 6.2 Ejecución nativa o contenedorizada

Todas las instancias usan Amazon Linux 2023 como sistema operativo. Los servicios del sistema (gateway, master y worker) no se ejecutan directamente sobre el sistema operativo, sino dentro de contenedores Docker. Esto garantiza que cada servicio lleve sus propias dependencias y se comporte igual en cualquier máquina. Al arrancar, cada instancia instala Docker y descarga la imagen de su servicio, publicada por la integración continua. En las instancias del gateway, además, un contenedor de nginx recibe el tráfico y lo pasa al servicio del gateway.

### 6.3 Instalación, configuración y puesta en marcha

En esta sección se describe el procedimiento para desplegar el sistema de archivos distribuido en AWS, usando la cuenta del Learner Lab. Toda la infraestructura está definida como código con Terraform, por lo que el despliegue no requiere crear recursos manualmente desde la consola: basta con preparar el entorno, configurar las credenciales y ejecutar unos pocos comandos. Los pasos siguientes cubren desde la instalación de las herramientas necesarias hasta la verificación del sistema, su uso desde el navegador y, finalmente, la eliminación de los recursos para no consumir el crédito de la cuenta.

**Paso 1. Preparar el entorno.** En el equipo desde el que se hará el despliegue se instalan Terraform (versión 1.6 o superior), la interfaz de línea de comandos de AWS (AWS CLI) y Git. Docker solo es necesario para ejecutar el sistema de forma local.

**Paso 2. Obtener el código.** Se clona el repositorio del proyecto y se entra a la carpeta `infra`, que contiene la definición de la infraestructura. No es necesario compilar el código, porque las imágenes ya están publicadas en Docker Hub.

**Paso 3. Configurar las credenciales de AWS.** En el portal del Learner Lab se inicia el laboratorio y, en la opción *AWS Details*, se copian las credenciales temporales. Se pegan en el archivo `~/.aws/credentials` del equipo y se verifica que funcionan con el comando `aws sts get-caller-identity`. Estas credenciales caducan al terminar la sesión del laboratorio, por lo que deben renovarse en cada sesión.

**Paso 4. Definir los parámetros del despliegue.** A partir del archivo de ejemplo `terraform.tfvars.example` se crea el archivo `terraform.tfvars`. En él se indican las imágenes Docker de cada componente, el número inicial de workers y si se usará un solo NAT Gateway para reducir costos. Este archivo no contiene información sensible: los secretos del sistema, como la clave de firma de los tokens JWT y el certificado TLS, los genera Terraform automáticamente durante el despliegue.

**Paso 5. Inicializar Terraform.** Se ejecuta `terraform init`, que descarga los proveedores necesarios para trabajar con AWS y para generar el certificado TLS.

**Paso 6. Revisar el plan.** Con `terraform plan` se obtiene una vista previa de todos los recursos que se crearán. Esto permite verificar la configuración antes de hacer cualquier cambio en la cuenta.

**Paso 7. Desplegar la infraestructura.** Con `terraform apply`, y confirmando con `yes`, se crea toda la infraestructura. El proceso tarda entre cinco y diez minutos y crea:

- una red privada virtual (VPC) distribuida en dos zonas de disponibilidad, `us-east-1a` y `us-east-1b`, con subredes públicas y privadas;
- las tablas de DynamoDB que almacenan los metadatos;
- dos masters en esquema activo–pasivo;
- los workers que almacenan los bloques de los archivos;
- un gateway por zona;
- un balanceador de carga público que recibe las conexiones por HTTPS.

Al terminar, cada servidor descarga su imagen Docker y arranca su servicio, lo que toma de dos a tres minutos adicionales.

**Paso 8. Verificar el funcionamiento.** Terraform entrega la dirección pública del sistema y su certificado TLS mediante los comandos `terraform output api_url` y `terraform output api_ca_pem`. El certificado se guarda en un archivo llamado `clawsha-ca.pem`. Para comprobar que el sistema está operativo se consulta la ruta `/healthz`, que debe responder `ok`. Después se consulta una ruta protegida del API, como `/api/v1/files`, que debe responder con el código 401, indicando que la petición llegó al gateway y que este exige autenticación.

**Paso 9. Usar el sistema.** Se abre la dirección pública en un navegador web. Como el certificado es autofirmado, el navegador muestra una advertencia de seguridad, que puede aceptarse o evitarse importando el archivo `clawsha-ca.pem` como certificado de confianza. Desde la interfaz web el usuario puede registrarse, iniciar sesión, crear carpetas y subir o descargar archivos. Para que otra persona use el sistema basta con compartirle la dirección pública y el archivo del certificado.

**Paso 10. Apagar el sistema.** Cuando ya no se necesita, se ejecuta `terraform destroy`, que elimina todos los recursos creados y evita consumir el crédito de la cuenta del Learner Lab.

## 6.4 Variables de entorno, puertos y dependencias

subsectionVariables de entorno

**Master.** Coordina el sistema y guarda los metadatos en DynamoDB.

Tabla 10: Variables de entorno del master

| Variable | Descripción | Valor en AWS |
|---|---|---|
| `MASTER_ID` | Identificador único del master | ID de la instancia EC2 |
| `MASTER_AZ` | Zona de disponibilidad donde corre | `us-east-1a` / `us-east-1b` |
| `MASTER_ADVERTISE_ADDR` | Dirección con la que se anuncia a los demás nodos | IP privada`:50051` |
| `MASTER_PORT` | Puerto del servicio gRPC | `50051` |
| `MASTER_HEALTH_PORT` | Puerto de los chequeos de salud | `8080` |
| `MASTER_REPLICATION` | Número de copias de cada bloque | `2` |
| `MASTER_BLOCK_SIZE` | Tamaño de los bloques en que se dividen los archivos | 16 MiB |
| `MASTER_TABLE_PREFIX` | Prefijo de las tablas de DynamoDB | `dfsha_` |
| `MASTER_CREATE_TABLES` | Si el master crea las tablas al iniciar | `false` (las crea Terraform) |
| `MASTER_DYNAMODB_ENDPOINT` | Dirección de una DynamoDB local | No se usa en AWS |
| `AWS_REGION` | Región de AWS | `us-east-1` |

**Worker.** Almacena los bloques de los archivos.

**Gateway.** Expone el API REST, autentica a los usuarios y mueve los bloques entre el cliente y los workers.

**Frontend (nginx).** Sirve la interfaz web y reenvía las peticiones del API al gateway.

Todos los componentes en Rust aceptan además `RUST_LOG`, que controla el nivel de detalle de los registros (por ejemplo, `info` o `debug`). Las credenciales de AWS que usa el master para acceder a DynamoDB no se configuran como variables: las proporciona automáticamente el rol de la instancia (`LabInstanceProfile`).

Tabla 11: Variables de entorno del worker

| Variable | Descripción | Valor en AWS |
|---|---|---|
| `WORKER_ID` | Identificador único del worker | ID de la instancia EC2 |
| `WORKER_AZ` | Zona de disponibilidad | `us-east-1a` / `us-east-1b` |
| `WORKER_PORT` | Puerto del servicio gRPC | `50052` |
| `WORKER_ADVERTISE_ADDR` | Dirección que el worker anuncia al master | IP privada`:50052` |
| `MASTER_ADDR` | Dirección del master | Balanceador interno, puerto `50051` |
| `WORKER_DATA_DIR` | Carpeta donde se guardan los bloques | `/data` (volumen EBS cifrado) |
| `WORKER_QUOTA_BYTES` | Espacio máximo que el worker ofrece | 9 GiB |
| `WORKER_FRAME_SIZE` | Tamaño de los fragmentos en que se transmite un bloque | 256 KiB |

Tabla 12: Variables de entorno del gateway

| Variable | Descripción | Valor en AWS |
|---|---|---|
| `GATEWAY_ADDR` | Dirección y puerto donde escucha | `0.0.0.0:3000` |
| `MASTER_ADDRESS` | Dirección del master | Balanceador interno, puerto `50051` |
| `JWT_SECRET` | Clave para firmar los tokens de sesión | Generada por Terraform |
| `CORS_ALLOWED_ORIGINS` | Orígenes web permitidos | No se usa en AWS (mismo origen) |

Tabla 13: Variables de entorno del frontend

| Variable | Descripción | Valor en AWS |
|---|---|---|
| `GATEWAY_UPSTREAM` | Dirección del gateway al que se reenvía `/api/` | `gateway:3000` |
| `VITE_API_BASE_URL` | URL del API (solo al compilar) | Vacía: misma dirección que la página |

### 6.5 Puertos

El único punto de entrada desde internet es el puerto 443 del balanceador público. El resto de los puertos solo son accesibles dentro de la red privada (VPC).

Tabla 14: Puertos utilizados por el sistema

| Puerto | Componente | Protocolo | Quién lo usa |
|---|---|---|---|
| 443 | Balanceador público | HTTPS (TLS 1.2/1.3) | Usuarios desde internet |
| 80 | nginx (frontend) | HTTP | Balanceador público |
| 3000 | Gateway | HTTP | Solo nginx, dentro de la misma instancia |
| 50051 | Master | gRPC | Gateways y workers, a través del balanceador interno |
| 8080 | Master | HTTP (`/health`, `/health/leader`) | Chequeos de salud del balanceador interno |
| 50052 | Worker | gRPC | Gateways (subida y descarga de bloques) y otros workers (replicación) |

No hay puertos de administración abiertos, como SSH (22). El acceso a los servidores se hace mediante AWS Systems Manager (*Session Manager*).

En la ejecución local los puertos se exponen así: frontend en 8088, gateway en 3001, masters en 50051/50052 (salud en 8081/8082), workers en 50053/50054 y DynamoDB Local en 8000.

## 7 Descripción del entorno nativo o Docker

### 7.1 Infraestructura y topología de despliegue

Toda la infraestructura de Clawsha se despliega en AWS (región `us-east-1`, dos zonas de disponibilidad) con Terraform, una herramienta de infraestructura como código. Terraform crea la VPC, las subredes públicas y privadas en las dos zonas, los *NAT Gateways*, los *Security Groups*, los balanceadores (NLB público e interno), los grupos de autoescalado y las instancias EC2, además de las tablas de DynamoDB y el *VPC endpoint* para llegar a ellas sin salir a Internet.

Los tipos de instancia se eligieron según la carga de cada componente:

| Componente | Instancia | Nodos | Subredes |
|---|---|---|---|
| NLB público y NAT | – | 1 NLB, 1 NAT por zona | 10.0.0.0/24, 10.0.1.0/24 (públicas) |
| Frontend (nginx) y gateway | `t3.small` | 2 (uno por zona) | 10.0.30.0/24, 10.0.31.0/24 |
| Master | `t3.small` | 2 (uno por zona) | 10.0.10.0/24, 10.0.11.0/24 |
| Worker | `t3.micro` + gp3 de 10 GB | 3 a 6 (ASG) | 10.0.20.0/24, 10.0.21.0/24 |

- **Gateway y master (`t3.small`)**: el gateway cifra y calcula el SHA-256 de cada byte que entra o sale y verifica contraseñas con argon2id; el master atiende todas las peticiones de metadatos y mantiene en memoria el mapa de ubicaciones de los bloques.

- **Worker (`t3.micro`)**: su trabajo es leer y escribir bloques, limitado por la red y el disco más que por la CPU. La capacidad se obtiene agregando workers al ASG en lugar de agrandar cada uno.

## 7.2 Ejecución nativa o contenedorizada

Todas las instancias usan Amazon Linux 2023 como sistema operativo. Los servicios del sistema (gateway, master y worker) no se ejecutan directamente sobre el sistema operativo, sino dentro de contenedores Docker. Esto garantiza que cada servicio lleve sus propias dependencias y se comporte igual en cualquier máquina. Al arrancar, cada instancia instala Docker y descarga la imagen de su servicio, publicada por la integración continua. En las instancias del gateway, además, un contenedor de nginx recibe el tráfico y lo pasa al servicio del gateway.

## 7.3 Instalación, configuración y puesta en marcha

En esta sección se describe el procedimiento para desplegar el sistema de archivos distribuido en AWS, usando la cuenta del Learner Lab. Toda la infraestructura está definida como código con Terraform, por lo que el despliegue no requiere crear recursos manualmente desde la consola: basta con preparar el entorno, configurar las credenciales y ejecutar unos pocos comandos. Los pasos siguientes cubren desde la instalación de las herramientas necesarias hasta la verificación del sistema, su uso desde el navegador y, finalmente, la eliminación de los recursos para no consumir el crédito de la cuenta.

**Paso 1. Preparar el entorno.** En el equipo desde el que se hará el despliegue se instalan Terraform (versión 1.6 o superior), la interfaz de línea de comandos de AWS (AWS CLI) y Git. Docker solo es necesario para ejecutar el sistema de forma local.

**Paso 2. Obtener el código.** Se clona el repositorio del proyecto y se entra a la carpeta `infra`, que contiene la definición de la infraestructura. No es necesario compilar el código, porque las imágenes ya están publicadas en Docker Hub.

**Paso 3. Configurar las credenciales de AWS.** En el portal del Learner Lab se inicia el laboratorio y, en la opción *AWS Details*, se copian las credenciales temporales. Se pegan en el archivo `~/.aws/credentials` del equipo y se verifica que funcionan con el comando `aws sts get-caller-identity`. Estas credenciales caducan al terminar la sesión del laboratorio, por lo que deben renovarse en cada sesión.

**Paso 4. Definir los parámetros del despliegue.** A partir del archivo de ejemplo `terraform.tfvars.example` se crea el archivo `terraform.tfvars`. En él se indican las imágenes Docker de cada componente, el número inicial de workers y si se usará un solo NAT Gateway para reducir costos. Este archivo no contiene información sensible: los secretos del sistema, como la clave de firma de los tokens JWT y el certificado TLS, los genera Terraform automáticamente durante el despliegue.

**Paso 5. Inicializar Terraform.** Se ejecuta `terraform init`, que descarga los proveedores necesarios para trabajar con AWS y para generar el certificado TLS.

**Paso 6. Revisar el plan.** Con `terraform plan` se obtiene una vista previa de todos los recursos que se crearán. Esto permite verificar la configuración antes de hacer cualquier cambio en la cuenta.

**Paso 7. Desplegar la infraestructura.** Con `terraform apply`, y confirmando con `yes`, se crea toda la infraestructura. El proceso tarda entre cinco y diez minutos y crea:

- una red privada virtual (VPC) distribuida en dos zonas de disponibilidad, `us-east-1a` y `us-east-1b`, con subredes públicas y privadas;
- las tablas de DynamoDB que almacenan los metadatos;
- dos masters en esquema activo–pasivo;
- los workers que almacenan los bloques de los archivos;
- un gateway por zona;
- un balanceador de carga público que recibe las conexiones por HTTPS.

Al terminar, cada servidor descarga su imagen Docker y arranca su servicio, lo que toma de dos a tres minutos adicionales.

**Paso 8. Verificar el funcionamiento.** Terraform entrega la dirección pública del sistema y su certificado TLS mediante los comandos `terraform output api_url` y `terraform output api_ca_pem`. El certificado se guarda en un archivo llamado `clawsha-ca.pem`. Para comprobar que el sistema está operativo se consulta la ruta `/healthz`, que debe responder `ok`. Después se consulta una ruta protegida del API, como `/api/v1/files`, que debe responder con el código 401, indicando que la petición llegó al gateway y que este exige autenticación.

**Paso 9. Usar el sistema.** Se abre la dirección pública en un navegador web. Como el certificado es autofirmado, el navegador muestra una advertencia de seguridad, que puede aceptarse o evitarse importando el archivo `clawsha-ca.pem` como certificado de confianza. Desde la interfaz web el usuario puede registrarse, iniciar sesión, crear carpetas y subir o descargar archivos. Para que otra persona use el sistema basta con compartirle la dirección pública y el archivo del certificado.

**Paso 10. Apagar el sistema.** Cuando ya no se necesita, se ejecuta `terraform destroy`, que elimina todos los recursos creados y evita consumir el crédito de la cuenta del Learner Lab.

## 7.4 Variables de entorno

**Master.** Coordina el sistema y guarda los metadatos en DynamoDB.

**Worker.** Almacena los bloques de los archivos.

**Gateway.** Expone el API REST, autentica a los usuarios y mueve los bloques entre el cliente y los workers.

**Frontend (nginx).** Sirve la interfaz web y reenvía las peticiones del API al gateway.

Todos los componentes en Rust aceptan además `RUST_LOG`, que controla el nivel de detalle de los registros (por ejemplo, `info` o `debug`). Las credenciales de AWS que usa el master para acceder a DynamoDB no se configuran como variables: las proporciona automáticamente el rol de la instancia (`LabInstanceProfile`).

## 7.5 Puertos

El único punto de entrada desde internet es el puerto 443 del balanceador público. El resto de los puertos solo son accesibles dentro de la red privada (VPC).

Tabla 16: Variables de entorno del master

| Variable | Descripción | Valor en AWS |
|---|---|---|
| MASTER_ID | Identificador único del master | ID de la instancia EC2 |
| MASTER_AZ | Zona de disponibilidad donde corre | us-east-1a / us-east-1b |
| MASTER_ADVERTISE_ADDR | Dirección con la que se anuncia a los demás nodos | IP privada:50051 |
| MASTER_PORT | Puerto del servicio gRPC | 50051 |
| MASTER_HEALTH_PORT | Puerto de los chequeos de salud | 8080 |
| MASTER_REPLICATION | Número de copias de cada bloque | 2 |
| MASTER_BLOCK_SIZE | Tamaño de los bloques en que se dividen los archivos | 16 MiB |
| MASTER_TABLE_PREFIX | Prefijo de las tablas de DynamoDB | dfsha_ |
| MASTER_CREATE_TABLES | Si el master crea las tablas al iniciar | false (las crea Terraform) |
| MASTER_DYNAMODB_ENDPOINT | Dirección de una DynamoDB local | No se usa en AWS |
| AWS_REGION | Región de AWS | us-east-1 |

Tabla 17: Variables de entorno del worker

| Variable | Descripción | Valor en AWS |
|---|---|---|
| WORKER_ID | Identificador único del worker | ID de la instancia EC2 |
| WORKER_AZ | Zona de disponibilidad | us-east-1a / us-east-1b |
| WORKER_PORT | Puerto del servicio gRPC | 50052 |
| WORKER_ADVERTISE_ADDR | Dirección que el worker anuncia al master | IP privada:50052 |
| MASTER_ADDR | Dirección del master | Balanceador interno, puerto 50051 |
| WORKER_DATA_DIR | Carpeta donde se guardan los bloques | /data (volumen EBS cifrado) |
| WORKER_QUOTA_BYTES | Espacio máximo que el worker ofrece | 9 GiB |
| WORKER_FRAME_SIZE | Tamaño de los fragmentos en que se transmite un bloque | 256 KiB |

Tabla 18: Variables de entorno del gateway

| Variable | Descripción | Valor en AWS |
|---|---|---|
| GATEWAY_ADDR | Dirección y puerto donde escucha | 0.0.0.0:3000 |
| MASTER_ADDRESS | Dirección del master | Balanceador interno, puerto 50051 |
| JWT_SECRET | Clave para firmar los tokens de sesión | Generada por Terraform |
| CORS_ALLOWED_ORIGINS | Orígenes web permitidos | No se usa en AWS (mismo origen) |

Tabla 19: Variables de entorno del frontend

| Variable | Descripción | Valor en AWS |
|---|---|---|
| GATEWAY_UPSTREAM | Dirección del gateway al que se reenvía /api/ | gateway:3000 |
| VITE_API_BASE_URL | URL del API (solo al compilar) | Vacía: misma dirección que la página |

Tabla 20: Puertos utilizados por el sistema

| Puerto | Componente | Protocolo | Quién lo usa |
|---|---|---|---|
| 443 | Balanceador público | HTTPS (TLS 1.2/1.3) | Usuarios desde internet |
| 80 | nginx (frontend) | HTTP | Balanceador público |
| 3000 | Gateway | HTTP | Solo nginx, dentro de la misma instancia |
| 50051 | Master | gRPC | Gateways y workers, a través del balanceador interno |
| 8080 | Master | HTTP (`/health`, `/health/leader`) | Chequeos de salud del balanceador interno |
| 50052 | Worker | gRPC | Gateways (subida y descarga de bloques) y otros workers (replicación) |

No hay puertos de administración abiertos, como SSH (22). El acceso a los servidores se hace mediante AWS Systems Manager (*Session Manager*).

En la ejecución local los puertos se exponen así: frontend en 8088, gateway en 3001, masters en 50051/50052 (salud en 8081/8082), workers en 50053/50054 y DynamoDB Local en 8000.

# 8 Pruebas y análisis de resultados

## 8.1 Plan y metodología de pruebas

Las pruebas se hicieron en dos niveles:

1. **Pruebas automatizadas.** La integración continua compila el proyecto, ejecuta `cargo clippy` con las advertencias tratadas como errores y corre `cargo test` en cada *pull request*, con una instancia de DynamoDB Local para las pruebas de la capa de datos. En la última ejecución sobre `main` (19 de octubre de 2026) pasaron **244 pruebas**: 133 del master (partición, ubicación, confirmación, *leases*, *fencing*, re-replicación, recolector de basura), 94 del worker (almacenamiento, *pipeline*, *scrubber*, *fencing*) y 17 del gateway (autenticación y rutas).
2. **Pruebas de sistema.** Se levantó el sistema completo con Docker Compose y se ejecutaron sobre él los escenarios del plan de pruebas: subidas y descargas reales, fallas provocadas y mediciones de rendimiento. Los resultados de esta sección provienen de ese nivel.

**Entorno de las pruebas de sistema.** El clúster local reproduce la topología de AWS con las mismas imágenes de los servicios (Tabla 21). HAProxy cumple el papel del NLB interno: revisa `/health/leader` cada 2 s y envía el tráfico gRPC al líder. Las zonas de disponibilidad son etiquetas: la AZ `a` tiene dos workers y la AZ `b` uno, como en la configuración mínima de AWS (3 workers, $2+1$). La caída de una AZ se simula deteniendo todos los contenedores de esa zona.

**Alcance y limitaciones de la medición.** Todos los contenedores comparten una sola máquina y se comunican por la red virtual de Docker, sin los límites de red de las instancias `t3` (RNF8, RNF9). Por eso, las cifras de rendimiento miden el costo del software (cifrado, hashes, *pipeline*,

Tabla 21: Entorno de las pruebas de sistema

| Elemento | Configuración |
|---|---|
| Máquina | Apple M5 Pro (15 núcleos), Docker Desktop con 15 CPU y 8 GB de RAM. |
| Plano de control | `master-a` (AZ a) y `master-b` (AZ b), HAProxy como NLB interno, DynamoDB Local en memoria. |
| Plano de datos | `worker-a` y `worker-c` (AZ a), `worker-b` (AZ b). Cuota de 2 GiB por worker en las pruebas de falla y de 8 GiB en las de rendimiento. |
| Acceso | Un gateway, accedido directamente por HTTP (sin nginx ni TLS). |
| Parámetros | $B = 16$ MiB, $R = 2$ salvo donde se indica; *scrubber* cada 30 s (en producción, 600 s). |
| Datos | Archivos aleatorios (`/dev/urandom`) de 0 B a 1 GiB. |
| Medición | Tiempos de `curl` desde la máquina anfitriona (`time_total` y `time_starttransfer`); integridad comparando el SHA-256 del archivo original con el descargado; estado del clúster con `GET /api/v1/cluster/status` y los registros de cada servicio. |

coordinación con el master) y son una cota superior; no representan el rendimiento esperado en AWS. Los escenarios que dependen de servicios de AWS (reemplazo de instancias por el ASG, conmutación entre gateways detrás del NLB público, TLS y escalado por la métrica de uso) no se ejecutaron en esta campaña.

## 8.2 Pruebas funcionales

**Integridad de extremo a extremo.** Se subió y descargó un archivo de cada tamaño crítico para la partición: vacío, un byte, exactamente un bloque, un bloque más un byte, un archivo de varios bloques con un bloque final parcial y un archivo de 1 GiB. En todos los casos el número de bloques coincidió con la partición de la Tabla 7 y el SHA-256 del archivo descargado fue igual al del original (Tabla 22).

Tabla 22: Prueba de integridad por tamaño de archivo

| Archivo | Bloques | Subida (s) | Descarga (s) | HTTP | SHA-256 |
|---|---|---|---|---|---|
| 0 B | 0 | 0,080 | 0,008 | 201 / 200 | igual |
| 1 B | 1 | 0,082 | 0,009 | 201 / 200 | igual |
| 16 MiB | 1 | 0,218 | 0,100 | 201 / 200 | igual |
| 16 MiB + 1 B | 2 | 0,211 | 0,151 | 201 / 200 | igual |
| 100 MiB | 7 | 0,949 | 0,620 | 201 / 200 | igual |
| 1 GiB | 64 | 9,349 | 5,777 | 201 / 200 | igual |

**Espacio de nombres, aislamiento y concurrencia.** La Tabla 23 reúne los demás casos funcionales. Todos dieron el resultado esperado.

Tabla 23: Casos de prueba funcionales

| ID | Caso | Esperado | Obtenido |
|---|---|---|---|
| F1 | `mkdir` de `/docs` y `/docs/sub` | 201 | 201, 201 |
| F2 | `mkdir` con el padre inexistente (`/no/parent`) | 404 | 404 |
| F3 | `mkdir` de un directorio que ya existe | 409 | 409 |
| F4 | `ls /docs` tras subir `a.bin` | el archivo y el subdirectorio | `a.bin` (1 B) y `sub` |
| F5 | `mv /docs/a.bin /docs/sub/b.bin` y descarga desde la ruta nueva y la vieja | 204; 200 con el mismo hash; 404 | 204; 200, hash igual; 404 |
| F6 | Borrar `/docs` sin `recursive` cuando no está vacío | 409 | 409 |
| F7 | `rm` de un archivo y `rm` recursivo de `/docs` | 204 y desaparecen del listado | 204, 204; no aparecen |
| F8 | Subir dos veces `/dup.bin` | la segunda queda como `dup (1).bin` | `/dup (1).bin`; el original intacto |
| F9 | Registrar un usuario existente | 409 | 409 |
| F10 | Registrar con contraseña de menos de 8 caracteres | 400 | 400 |
| F11 | Inicio de sesión con contraseña incorrecta | 401 | 401 |
| F12 | El usuario `bob` descarga, lista y borra archivos de `alice` | 404 sin revelar nada; nada cambia | 404; listado vacío; 404; `alice` sigue descargando |
| F13 | Dos subidas de 100 MiB a `/race.bin` con 50 ms de diferencia | una 201 y la otra 409 | 201 (1,08 s) y 409 (8 ms); un solo archivo |

En F13, la segunda subida encuentra la ruta tomada por el *lease* de la primera y se rechaza antes de enviar datos, así que nunca hay dos escritores sobre la misma ruta (RF3).

## 8.3 Pruebas de rendimiento y escalabilidad

Cada medición se repitió 5 veces sobre un clúster recién creado; se reporta la mediana y, entre corchetes, el mínimo y el máximo. Las subidas se miden de punta a punta (incluyen el cifrado, las dos réplicas y la confirmación en el master) y todas las descargas se verificaron con SHA-256. La máquina anfitriona se seguía usando durante las pruebas, así que entre repeticiones hubo variaciones de hasta ±15 % (más en casos aislados). Por eso solo se interpretan las diferencias mayores que ese margen. Como se explicó en la metodología, estas cifras son una cota superior y no el rendimiento esperado en AWS.

**Throughput según el tamaño del archivo.** La Figura 8 y la Tabla 24 muestran el throughput de un solo cliente. Con archivos de un bloque, el costo fijo de cada subida (plan, *lease* y confirmación en DynamoDB) pesa más; a partir de 100 MiB, la subida se estabiliza en unos 125–135 MiB/s y la descarga en 170–200 MiB/s. La descarga es más rápida porque no escribe en disco ni confirma metadata, aunque lee un bloque a la vez.

**Clientes concurrentes.** Se lanzaron 1, 5 y 10 clientes a la vez, cada uno con un archivo distinto de 100 MiB: primero todas las subidas y luego todas las descargas. Se repitió 5 veces cada nivel

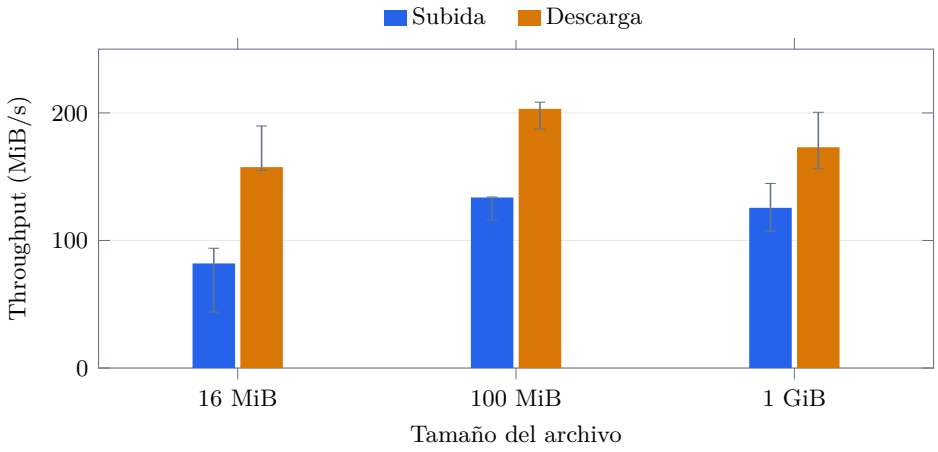



Figura 8: Throughput de un cliente según el tamaño del archivo (mediana de 5 repeticiones; las barras de error marcan el mínimo y el máximo). Clúster local, $B = 16$ MiB, $R = 2$.

Tabla 24: Throughput de un cliente según el tamaño del archivo (MiB/s)

| Archivo | Bloques | Subida | Descarga |
|---|---|---|---|
| 16 MiB | 1 | 82,0 [43,9–93,9] | 157,5 [154,9–189,8] |
| 100 MiB | 7 | 133,7 [115,9–134,1] | 203,2 [187,4–208,4] |
| 1 GiB | 64 | 125,6 [107,4–144,7] | 173,1 [156,2–200,5] |

(Figura 9). El throughput total sube de 1 a 5 clientes y luego se aplana: con un solo gateway, que cifra y calcula los hashes de todos los bytes, y con todos los servicios en la misma máquina, el sistema se satura alrededor de 200–240 MiB/s. Con 10 clientes, cada uno obtuvo en promedio unos 23 MiB/s de subida y 21 MiB/s de descarga. Las 80 subidas y las 80 descargas de las 5 repeticiones terminaron bien y con el hash correcto.

**Tamaño de bloque.** Se subió y descargó un archivo de 256 MiB con bloques de 4, 16 y 64 MiB (Tabla 25). En local las diferencias son pequeñas: los bloques de 4 MiB fueron algo más rápidos, porque con 4 bloques en vuelo el *pipeline* se llena antes y el gateway trabaja con menos memoria. Los bloques grandes no dieron ventaja, ya que en una sola máquina no hay latencia de red que amortizar. La elección de 16 MiB se mantiene por los motivos de la Sección 5: con 4 MiB habría 4 veces más filas en DynamoDB y entradas en el mapa del líder, y el tope de 65 536 bloques limitaría los archivos a 256 GiB.

Tabla 25: Archivo de 256 MiB con distintos tamaños de bloque (MiB/s)

| Bloque | Bloques | Subida | Descarga |
|---|---|---|---|
| 4 MiB | 64 | 134,8 [129,6–136,0] | 216,4 [207,2–240,9] |
| 16 MiB | 16 | 128,2 [108,3–133,4] | 177,5 [157,3–181,6] |
| 64 MiB | 4 | 114,1 [108,9–115,3] | 166,8 [153,3–199,0] |

**Costo de replicar.** Se comparó la subida con $R = 1$ y con $R = 2$ (Tabla 26). Con 100 MiB, la segunda réplica costó un 9 % de throughput; con 1 GiB la diferencia quedó dentro de la variación

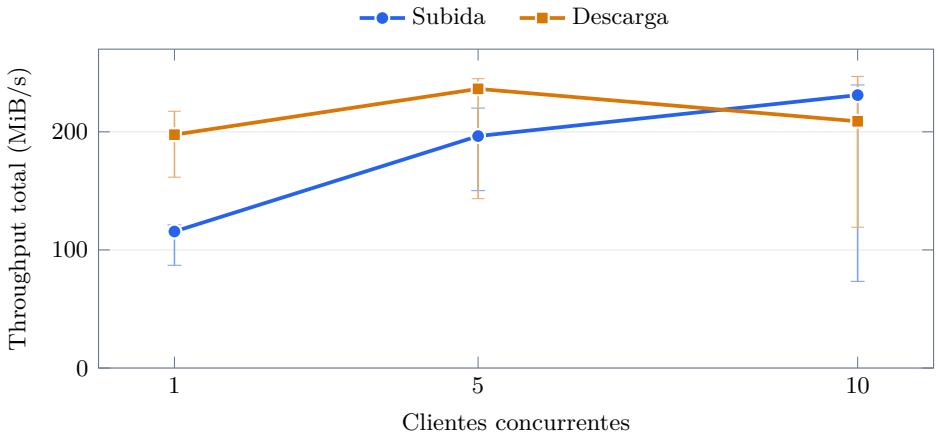



Figura 9: Throughput total con varios clientes de 100 MiB cada uno (mediana de 5 repeticiones; las barras marcan el mínimo y el máximo). Con 10 clientes, tres repeticiones dieron 231–240 MiB/s de subida y dos quedaron muy por debajo (137 y 73 MiB/s), lo que refleja la variabilidad de la máquina anfitriona.

entre repeticiones. El costo es bajo porque el primer worker reenvía cada *frame* al segundo mientras lo recibe, en vez de esperar al bloque completo. En AWS el costo será mayor: con $R = 2$ cada byte debe entrar a las dos AZ, así que el límite pasa a ser la red de la AZ con menos workers (RNF8). La descarga no depende de $R$ (se lee una sola copia); sus diferencias en la tabla son una medida del ruido del entorno.

Tabla 26: Subida y descarga con una y dos réplicas (MiB/s)

| **Archivo** | $R$ | **Subida** | **Descarga** |
|---|---|---|---|
| 100 MiB | 1 | 138,5 [109,3–139,0] | 223,0 [194,2–224,6] |
| 100 MiB | 2 | 126,0 [125,7–132,0] | 190,2 [180,4–203,1] |
| 1 GiB | 1 | 104,4 [77,0–148,8] | 140,4 [106,3–173,0] |
| 1 GiB | 2 | 121,8 [109,6–141,8] | 164,4 [146,8–177,6] |

**Latencia.** La Tabla 27 contrasta las latencias medidas desde el cliente con los objetivos de RNF10. Las operaciones de metadata son una llamada gRPC al líder y pocas operaciones en DynamoDB, y quedan muy por debajo del objetivo. El primer byte de una descarga se midió como el tiempo hasta recibir el primer byte del cuerpo (no de las cabeceras, que el gateway envía antes de leer ningún bloque): 39 ms en p95 para un bloque completo de 16 MiB, que el gateway debe recibir y verificar entero antes de entregar. En AWS, ese tiempo lo dominará la red del worker ($\approx$ 2,1 s a 7,6 MiB/s).

**Escalabilidad: un worker nuevo en un clúster cargado.** Para comprobar que AvailableSpace dirige los bloques nuevos hacia un worker vacío, se cargó el clúster con 15 archivos de 100 MiB (105 bloques) y luego se agregó `worker-d`, vacío, en la AZ `b`, con la misma cuota que los demás. A continuación se subieron 20 archivos más (140 bloques, 280 copias). La Tabla 28 muestra el resultado.

Cada bloque dejó exactamente una copia en cada AZ (140 en `a` y 140 en `b`), y dentro de la AZ `b` el

Tabla 27: Latencia medida en el clúster local frente al objetivo de RNF10

| Operación | Muestras | p50 | p95 | Objetivo p95 |
|---|---|---|---|---|
| `ls` | 100 | 1,9 ms | 2,4 ms | 100 ms |
| `mkdir` | 100 | 2,8 ms | 3,6 ms | 100 ms |
| `mv` | 100 | 3,8 ms | 4,9 ms | 100 ms |
| `rm` | 100 | 4,1 ms | 5,4 ms | 100 ms |
| Primer byte, archivo de 1 MiB | 30 | 7,0 ms | 11,4 ms | 300 ms |
| Primer byte, bloque de 16 MiB | 30 | 29,7 ms | 38,6 ms | 3 s |

Tabla 28: Reparto de 140 bloques nuevos tras agregar un worker vacío

| Worker | AZ | Uso antes | Copias nuevas | Parte de su AZ | Uso después |
|---|---|---|---|---|---|
| `worker-a` | a | 8,3 % | 68 | 49 % | 19,4 % |
| `worker-c` | a | 7,1 % | 72 | 51 % | 19,1 % |
| `worker-b` | b | 15,4 % | 57 | 41 % | 26,4 % |
| `worker-d` | b | 0 % | 83 | 59 % | 12,1 % |

worker nuevo recibió el 59 % de las copias, frente al 50 % de una elección uniforme. Esto coincide con el cálculo de la Sección 5.2: el worker vacío recibe la primera réplica en ≈ 30 % de los bloques (participa en la comparación con probabilidad 2/4 y gana con 0,6) y, cuando la primera cae en la AZ `a`, gana la segunda el 60 % de las veces, lo que da ≈ 58 %. Los workers de la AZ `a`, con usos parecidos (menos de 5 puntos de diferencia), se repartieron sus copias casi por mitades. El worker nuevo se llena más rápido sin mover los bloques existentes, pero el sesgo es moderado: con este ritmo tarda varias cargas en alcanzar el uso de los demás.

## 8.4 Pruebas de disponibilidad, consistencia y seguridad

Cada escenario provoca una falla real (corromper un archivo en el disco de un worker o detener contenedores) y verifica dos cosas: que los archivos confirmados se sigan leyendo con el mismo SHA-256 y que el sistema vuelva solo a tener dos copias de cada bloque. Los tiempos se miden desde el momento en que se provoca la falla.

**D1. Réplica corrupta.** Se subió un archivo de 16 MiB (un bloque, con réplicas en `worker-b` y `worker-c`) y se sobrescribió con ceros el primer MiB de la copia de `worker-b`. Las descargas siguientes salieron bien, servidas por la copia sana. En su siguiente pasada, el *scrubber* de `worker-b` detectó el hash distinto, puso la copia en cuarentena y la reportó al master (≈ 15 s); 14,6 s después el líder ordenó copiarla desde `worker-c` (10 s de gracia más el intervalo del ciclo). La copia reparada quedó con el mismo SHA-256 que la sana.

**D2. Conmutación de réplica en la lectura.** Para forzar la lectura de copias dañadas, se corrompió la copia de la AZ `b` de los 7 bloques de un archivo de 100 MiB y se descargó de inmediato. La descarga terminó con el SHA-256 correcto: en 2 de los 7 bloques el gateway pidió primero la copia dañada, `worker-b` respondió `DATA_LOSS` sin enviar datos y el gateway pasó a `worker-a`. El *scrubber* puso en cuarentena 3 copias en la pasada que estaba en curso y las 4 restantes en la siguiente (se dañaron mientras la pasada ya las había revisado). Las 7 quedaron reparadas unos

60 s después de la corrupción y las pasadas siguientes no encontraron ninguna copia dañada.

**D3. Worker caído y su regreso.** Se detuvo `worker-c`, que tenía 46 bloques. La Tabla 29 muestra la secuencia. Una descarga de 1 GiB lanzada justo después de la caída terminó bien (5,4 s): el gateway saltó la réplica que no respondía.

Tabla 29: Secuencia tras detener `worker-c` (D3)

| Tiempo | Evento |
|---|---|
| 0 s | Se detiene `worker-c`. |
| 9,8 s | El master lo marca SUSPECT: deja de asignarle bloques nuevos. |
| 30,5 s | Lo marca DEAD: 46 bloques quedan sub-replicados. |
| 47,2 s | Sub-replicados en 0: las 46 copias se rehicieron en `worker-a` (AZ a, la que perdió la copia). |
| 0 s | `worker-c` vuelve con sus 46 bloques: esos bloques tienen 3 copias. |
| ≈ 47 s | El recolector de basura borra 45 copias sobrantes; cada bloque vuelve a tener exactamente 2. |

Copiar los 46 bloques (≈ 0,7 GiB) tomó menos de 17 s con el límite de 2 copias simultáneas por worker. El criterio del plan de pruebas era volver a cero sub-replicados en unos 60 s; se logró en 47 s, de los cuales 30 s son la espera deliberada antes de declarar muerto al worker.

**D4. Caída de una zona de disponibilidad.** Se detuvieron a la vez, sin apagado ordenado, `master-b` (que era el líder) y `worker-b`, es decir, toda la AZ `b`:

- A los 15,8 s, `master-a` ya era líder (*fencing token* 2) y las operaciones de metadata respondían. A los 22,6 s, cuando terminó de reconstruir el mapa de ubicaciones con los *block reports*, volvieron las descargas.
- Los 8 archivos confirmados (de 1 B a 1 GiB) se descargaron con el SHA-256 correcto: cada bloque tenía su otra copia en la AZ `a`.
- Al salir del modo seguro, los 92 bloques que habían quedado con una sola copia se replicaron entre `worker-a` y `worker-c` en unos 10 s (modo degradado: dos copias en la misma AZ). Una subida durante la caída también funcionó.
- Al volver la AZ `b`, `master-b` se unió como standby y el recolector de basura borró las copias sobrantes de la AZ `a`.

**D5. Caída del líder durante una subida.** Se mató al líder 3,5 s después de iniciar una subida de 1 GiB. La subida falló con 502 a los 10,7 s, cuando el gateway intentó confirmarla sin líder, y el archivo no apareció en el listado: no quedó ningún archivo parcial visible (RF5). El otro master atendió a los 17,2 s (*fencing token* 3) y los archivos confirmados antes de la falla siguieron intactos.

**D6. Caída del líder durante una descarga.** Se mató al líder 1,5 s después de iniciar la descarga de un archivo de 1 GiB. La descarga terminó con 200 en 4,3 s y el SHA-256 correcto: una vez que el gateway tiene el plan de lectura, ya no necesita al master.

**Pruebas de seguridad.** La Tabla 30 resume las pruebas sobre la autenticación y la protección de la web. Para los casos de JWT, los tokens se fabricaron a mano con el mismo secreto del

gateway, de modo que solo cambia la condición que se prueba; un token fabricado igual pero vigente fue aceptado (control).

Tabla 30: Pruebas de seguridad

| ID | Caso | Esperado | Obtenido |
|---|---|---|---|
| S1 | Petición sin token | 401 | 401 |
| S2 | Token con la firma alterada | 401 | 401 |
| S3 | Token vencido hace 1 s, firmado con el secreto correcto | 401 | 401 |
| S4 | Token vigente firmado con otro secreto | 401 | 401 |
| S5 | Token con `alg: none` y sin firma | 401 | 401 |
| S6 | Control: token fabricado con el secreto correcto y vigente | 200 | 200 |
| S7 | Ruta inexistente del gateway | 404 | 404 |
| S8 | Subida a `/../../etc/x` | rechazo; nada escrito fuera de `/data` | 400; 0 archivos fuera de las carpetas del worker |
| S9 | Archivo HTML pedido con `disposition=inline` | forzar descarga | `attachment`, `nosniff` |

## 8.5 Análisis de resultados y limitaciones

**Lo que se cumplió.** En todas las pruebas, los archivos confirmados se leyeron con el SHA-256 correcto, incluso con una réplica corrupta, un worker caído, una AZ completa caída o el líder muerto a mitad de una operación. El sistema volvió solo a tener dos copias de cada bloque en menos de un minuto en todos los casos, sin intervención manual, y el recolector de basura eliminó las copias sobrantes cuando los nodos volvieron. La partición se comportó como se diseñó en los casos límite (0 B, un bloque exacto, un bloque más un byte) y la política de ubicación cumplió sus dos promesas: una copia por AZ y un sesgo medible hacia el worker más vacío.

**Tiempo de *failover*.** El sistema volvió a atender a los 15,8–17,2 s de matar al líder, dentro del límite de 30 s de RF9. El tiempo se compone de la expiración del *lease* de líder (hasta 10 s más el intervalo de renovación de 3 s), dos chequeos consecutivos de HAProxy cada 2 s para dar por sano al nuevo líder y la reconexión del gateway. Las descargas tardaron unos segundos más (22,6 s) porque el nuevo líder necesita los *block reports* para saber dónde está cada bloque. El tiempo se puede bajar acortando el intervalo de chequeo del balanceador, a costa de más tráfico de control.

**Rendimiento.** Con un solo cliente, el software procesa unos 125 MiB/s de subida y 170 MiB/s de descarga en local, incluyendo cifrado, hashes y dos réplicas. Eso es más de 15 veces la red base de un `t3.micro` (≈ 7,6 MiB/s), así que en AWS el cuello de botella será la red de las instancias y no el código, como suponen RNF8 y RNF9. Con 10 clientes concurrentes, cada uno obtuvo más de 20 MiB/s, por encima del objetivo de 5 MiB/s por cliente. Esto no verifica los objetivos de AWS, que deben medirse en el despliegue real, pero indica que el software no los impide. Las latencias de metadata quedaron dos órdenes de magnitud por debajo de RNF10.

**Limitaciones encontradas.** Las pruebas revelaron cinco comportamientos que conviene corregir o documentar:

1. **El gateway no reporta las copias que el worker rechaza.** Cuando el worker detecta el hash distinto y responde `DATA_LOSS`, el gateway cambia de réplica correctamente pero no llama `ReportCorruption`, porque solo reporta los hashes distintos que detecta él mismo. La copia dañada se repara igual, pero hasta un intervalo del *scrubber* después (600 s en producción) en vez de en segundos (D2). Basta con tratar `DATA_LOSS` como corrupción en el gateway.
2. **La capacidad útil la fija la AZ más pequeña.** Como cada bloque tiene una copia en cada AZ, con 3 workers $(2+1)$ el único worker de la AZ `b` guarda una copia de todos los bloques. Cuando se llena, las subidas fallan con 503 aunque la AZ `a` tenga espacio; así ocurrió en las pruebas de falla, con cuotas de 2 GiB. Con 4 workers $(2+2)$ la capacidad se duplica. Además, el error debería indicar falta de espacio y no una indisponibilidad genérica.
3. **Una subida cortada por un *failover* retiene su espacio una hora.** Si el líder cae durante una subida, el gateway no logra abortarla y la versión queda PENDING. Sus bloques ocupan espacio hasta que el recolector la descarta (1 h, más 5 min de gracia), lo que contribuyó a llenar la AZ `b` en las pruebas.
4. **El recolector no rebalancea.** Al quitar las copias sobrantes elige la AZ con más copias, pero no el worker más lleno dentro de ella: tras la prueba D4, `worker-a` quedó con 2 bloques y `worker-c` con 94. No se pierde nada, pero los bloques nuevos tardan en compensar el desbalance, porque AvailableSpace es un sesgo moderado.
5. **Las descargas leen un bloque a la vez.** Cada descarga está limitada por la red de un solo worker en cada momento (RNF9). La lectura anticipada de varios bloques en paralelo y las lecturas *hedged* del plan de diseño no se implementaron.

**Lo que no se probó.** Por depender de servicios de AWS, en esta campaña no se ejecutaron el reemplazo de instancias por el ASG, la conmutación entre gateways detrás del NLB público, TLS, el escalado por la métrica `ClusterUsedPercent` ni las pruebas con la red limitada. Tampoco se midió el throughput con los límites de red reales de las instancias `t3`.

# 9 Conclusiones

**Separar el plano de control del plano de datos y replicar por zona hizo al sistema tolerante a fallas reales.** Siguiendo el modelo de HDFS y GFS [**?**, **?**], el master solo maneja metadata y los bloques viajan directamente entre el gateway y los workers, con dos copias en zonas de disponibilidad distintas. En las pruebas, ningún archivo confirmado se perdió ni se leyó con un hash distinto, ni siquiera al corromper réplicas en disco, detener un worker, apagar una zona completa o matar al líder a mitad de una operación. El sistema volvió solo a tener dos copias de cada bloque en menos de un minuto, y el *failover* del master (15,8–17,2 s) quedó dentro del límite de 30 s. El *lease* de líder con *fencing token* [**?**] y la confirmación atómica en DynamoDB evitaron que un líder obsoleto o una subida interrumpida dejaran metadata inconsistente.

**El tamaño de bloque y la política de ubicación son un equilibrio, no un óptimo.** Los bloques de 16 MiB reparten entre varios workers incluso los archivos medianos y mantienen acotadas la memoria del gateway y la metadata, aunque en local los bloques de 4 MiB fueron algo más rápidos. La regla AvailableSpace de HDFS [5], una variante de la elección entre dos candidatos al azar [6], garantizó una copia por zona y dirigió el 59 % de las copias nuevas hacia

un worker vacío, sin mover los bloques existentes. A cambio, la capacidad útil queda fijada por la zona con menos workers y el sistema no rebalancea por sí solo; ambas son decisiones conscientes que conviene revisar si el clúster crece.

**Probar con fallas provocadas fue tan importante como implementar.** Las pruebas de sistema confirmaron los requerimientos, pero también revelaron comportamientos que el diseño no anticipaba: el gateway no reporta las copias que el propio worker rechaza, una subida cortada por un *failover* retiene su espacio durante una hora y las descargas leen un bloque a la vez. En rendimiento, el software procesó unos 125 MiB/s de subida en una sola máquina, más de 15 veces la red base de un `t3.micro`, de modo que en AWS el cuello de botella será la red y no el código. Validar esas cifras en el despliegue real y corregir las limitaciones encontradas es el trabajo que queda. Además, construir el sistema en Rust con un contrato gRPC único [7, 8, 10] permitió que tres personas integraran sus componentes en paralelo sin romperse entre sí.

## Referencias


[1] Universidad EAFIT, *SI3007 Sistemas Distribuidos – Proyecto 1: Diseño e implementación de un sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad (DFSha)*, enunciado del curso, 2026-2.

[2] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. distributed-systems.net, 2023. https://www.distributed-systems.net/

[3] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[4] The Apache Software Foundation, "HDFS Architecture," documentación de Apache Hadoop. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[5] The Apache Software Foundation, "hdfs-default.xml" (parámetros `dfs.namenode.available-space-block-placement-policy.*`), documentación de Apache Hadoop. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml

[6] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, 2001.

[7] S. Klabnik y C. Nichols, *The Rust Programming Language*. https://doc.rust-lang.org/book/

[8] Tokio, "Tokio tutorial." https://tokio.rs/tokio/tutorial

[9] Tokio, "axum," documentación del *crate*. https://docs.rs/axum

[10] gRPC Authors, "gRPC documentation." https://grpc.io/docs/

[11] Google, "Protocol Buffers documentation." https://protobuf.dev/

[12] Amazon Web Services, "Amazon DynamoDB Developer Guide." https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/

[13] Amazon Web Services, "User Guide for Network Load Balancers." https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

[14] Amazon Web Services, "Amazon EC2 Auto Scaling User Guide." https://docs.aws.amazon.com/autoscaling/ec2/userguide/

[15] HashiCorp, “Terraform documentation.” https://developer.hashicorp.com/terraform/docs

[16] HashiCorp, “AWS Provider,” Terraform Registry. https://registry.terraform.io/providers/hashicorp/aws/latest/docs

[17] Docker Inc., “Docker documentation” y “Docker Compose.” https://docs.docker.com/

[18] HAProxy Technologies, “HAProxy documentation.” https://docs.haproxy.org/